

CardBenePICK

복잡한 카드 혜택 조건을 AI가 분석하여
최적의 카드를 추천하는 All-in-One 핀테크 솔루션

우리 FISA 최종 프로젝트

AI 엔지니어링 5팀

팀명: 베테크

팀원: 이계무, 김유라, 남상원, 오윤진, 최수빈

멘토: 정상원 멘토님



CONTENTS

- 01 — 팀 소개 및 프로젝트 수행 과정
- 02 — 프로젝트 개요
- 03 — 데이터 및 아키텍처 구성
- 04 — 서비스 구성 및 구현
- 05 — 프로젝트 결론 및 기대효과

01

팀 소개 및 프로젝트 수행 과정

1-1 ————— 팀원 소개 및 역할 분배

1-2 ————— 수행 절차

1-3 ————— 협업과정

팀원 소개 및 역할 분배



이계무 (PM)

MCP

AI Agent
AWS 배포



김유라

Data

Frontend
Backend



남상원

ML Modeling

Data
ML Backend API



오윤진

AI Agent

Backend
Airflow
Docker
Frontend



최수빈

RAG

Frontend
Backend
Data

수행 절차

Agent Public

Python • 0 • 0 • 0 • 0 • Updated 37 minutes ago

cardbenepick-app Public

TypeScript • 0 • 0 • 0 • 0 • Updated 46 minutes ago

model-server Public

Python • 0 • 0 • 0 • 0 • Updated 2 days ago

airflow Public

카드베네픽에서 실행되는 배치성 작업을 airflow를 통해 자동화합니다

0 • 0 • 0 • 0 • Updated 2 days ago

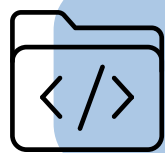
Modeling Private

Jupyter Notebook • 0 • 0 • 0 • 0 • Updated 3 days ago

.github Public

[CardBenePICK] GitHub Organization 공용 설정

0 • 0 • 5 • 0 • Updated 3 weeks ago



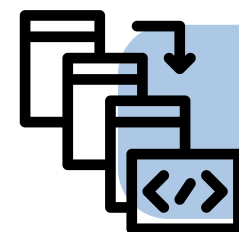
Organization

핵심 협업 원칙

1. **dev** 브랜치 타겟: 모든 PR은 **dev** 브랜치를 타겟으로 합니다. (**main** 브랜치로 직접 PR 금지)
2. **코드 리뷰**: 모든 코드는 최소 1명 이상의 리뷰어에게 승인(Approve)을 받아야 Merge할 수 있습니다.
3. **컨벤션 준수**: 브랜치, 커밋, PR 생성 시 정해진 컨벤션을 따릅니다.

1. 브랜치 전략

- main**:
 - [운영/배포] 브랜치입니다.
 - 오직 **dev** 브랜치에서 정식 릴리즈된 버전만 Merge 됩니다.
 - 절대 직접 커밋하거나 이 브랜치로 PR을 보내지 않습니다.
- dev**:
 - [개발/통합] 브랜치입니다.
 - 다음 릴리즈를 준비하는 모든 기능이 통합되는 메인 개발 브랜치입니다.
 - 모든 **feature** 브랜치는 **dev**를 타겟으로 PR을 보냅니다.
- feature/[description]**:
 - [기능 개발] 브랜치입니다.
 - 반드시 **dev** 브랜치에서 생성합니다.
 - 기능 설명은 간결한 영어로 작성합니다. (예: `feature/login-api`)
- fix/[description]**:
 - [버그 수정] 브랜치입니다.
 - dev** 브랜치에서 발생한 버그를 수정할 때 사용합니다. (예: `fix/login-validation`)
- hotfix/[description]**:
 - [긴급 수정] 브랜치입니다.
 - main** 브랜치에서 직접 생성하며, 수정 완료 후 **main** 과 **dev** 브랜치 모두에 Merge해야 합니다.



Branch

회의록

표

날짜	이름
2025년 10월 17일	10-17
2025년 10월 18일	10-18 강의 + 기획
2025년 10월 18일	10-17~18 기획 + 멘토링 정리
2025년 10월 21일	10-21 문제 인식 및 아이디어 회의
2025년 10월 22일	10-22 기능 추가 아이디어
2025년 10월 24일	10-23~4 프로젝트 산출물 작성
2025년 10월 27일	10-27 기획 시나리오
2025년 10월 28일	10-28 테스트 기능을 위한 데이터 정리
2025년 10월 29일	데이터(더미생성, 전처리), test rag



일간 회의

수행 절차

구분	기간	활동	비고
사전 기획	10/17(금) ~ 10/24(금)	카드 소비 관련 뉴스 / 핀테크 서비스 분석	‘카드베네퍹’ 아이디어 선정
데이터 수집	10/25(토) ~ 11/7(금)	마이데이터 API 분석 및 관련 더미데이터 생성	
데이터 전처리	10/25(월) ~ 11/14(금)	우리카드 사용내역 데이터 EDA 및 전처리	
AI 서비스 개발/모델링	11/1(월) ~ 12/5(금)	LLM & MCP 서버 구성, MCP Tool 제작, 모델선정	팀별 중간보고 실시
서비스 구축 및 배포	11/11(화) ~ 12/5(금)	React, TypeScript 사용 서비스 구성 Agent와 ML 모델 서비스에 적용	최적화, 오류 수정
총 개발기간	10/18(월) ~ 12/5(금)(총 7주)		

프로젝트 개요

2-1 — 주제 선정 배경

주제 선정 배경



신한카드 Mr.Life

Essential For Every Lifestyle

월납요금 10% 할인

Time 10% 할인

주말 주유소 60원/ℓ 할인

카드 디자인



온라인쇼핑

온라인 쇼핑 10% 할인

오후 9시 ~ 오전 9시까지 Night 할인서비스 10% 할인

온라인 쇼핑 10% 할인

- 옥션, G마켓, AK물, 11번가, 티켓몬스터, 위메프, 쿠팡
- 서비스 영역별 각각 일 1회 할인 적용
- 1회 이용 금액 1만원까지 할인 적용 (1회 최대 1천원 할인)
- 월 10회 할인 적용

1회 최대 1천원 할인

TIME 할인서비스 할인한도안내

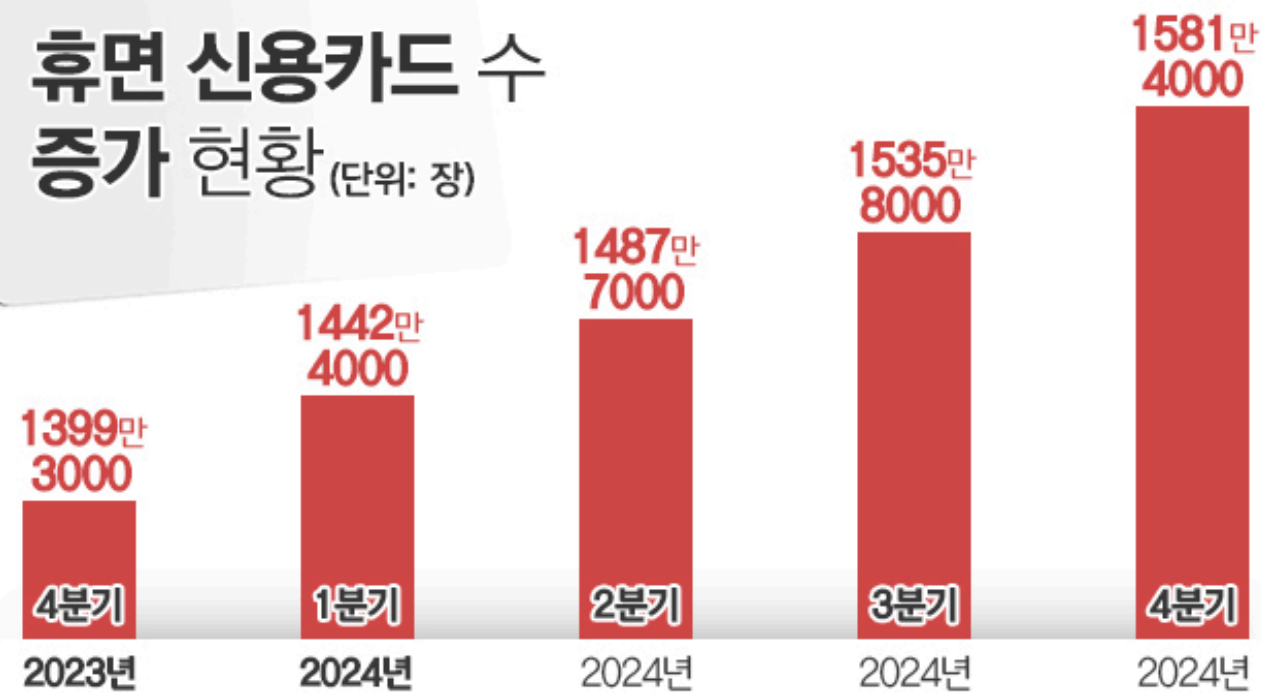
- 전월실적 30만원~50만원 : 1만원
- 전월실적 50만원 ~100만원 : 2만원
- 전월실적 100만원 이상 : 3만원

유의사항

- 서비스 대상 거래건 중 신한카드 전표 매입 순서대로 결제일 할인이 적용됩니다.
- TIME 할인서비스는 전월 이용금액에 따라 제공된 월 할인 한도 내에서 서비스가 제공됩니다.
- 신규 발급 회원에 대해서는 카드사용 등록월의 익월말(등록월+1개월)까지 1만원의 할인한도가 제공됩니다.
- 편의점, 병원/약국, 세탁소, 식음료는 신한카드 가맹점 업종 기준으로 할인이 제공됩니다.
- Time 할인서비스는 주중/주말 상관없이 제공되는 서비스이며, Night Time 할인서비스는 승인시간 기준으로 오후 9시부터 오전 9시까지 제공됩니다.
- 병원/약국 10% 할인서비스 대상 가맹점에서 동물병원은 제외되며, 치과/한의원 포함 제공됩니다.
- 티켓몬스터, 위메프, 쿠팡 사이트 직접 접속 시 혹은 대상가맹점 앱을 통해서 접속 시에만 할인이 적용됩니다.
- 전월 이용실적은 일시불+할부 금액 기준이며 (전월 할인거래 실적 포함) 교통이용금액은 전전월 이용금액 기준, 해외이용금액은 매입일자를 기준으로 적용됩니다.
- 월 기준 : 매월 1일 ~ 말일

카드 별 다양한 종류의 혜택과 한도, 복잡한 실적 구조

주제 선정 배경

휴면 신용카드 수
증가 현황 (단위: 장)

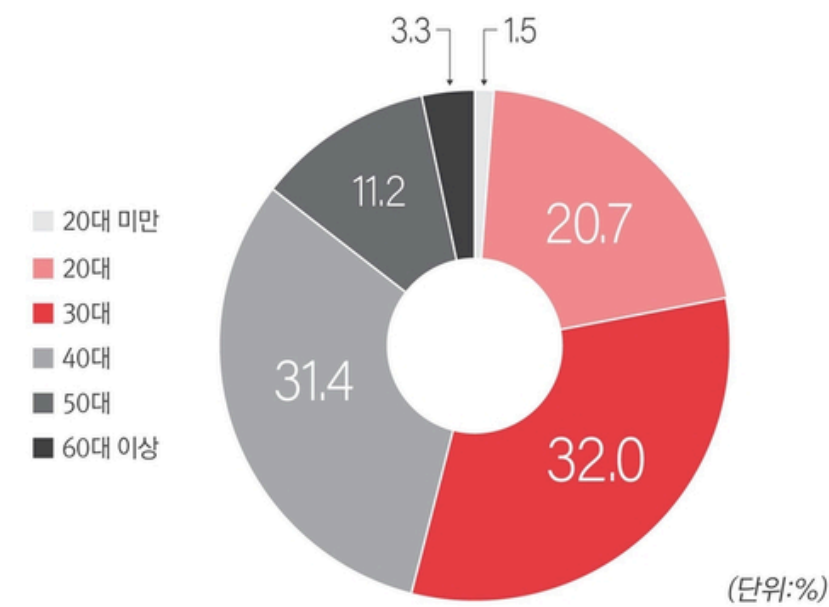
자료: 여신금융협회

휴면 신용카드 수의 증가



MZ세대, 베테크(Benefit+재테크)에 눈뜨다

'21년도 전체 마이태그 이용고객 중 20-40대 고객 비중



(단위: %)

출처: BC카드

베테크에 대한 관심 증대

주제 선정 배경

	결제 편의성	자산/소비 관리	혜택 요약. 피킹룰	결제 순간 최적 카드 추천	최적 혜택 카드 선택
 SAMSUNG Pay	○	×	×	×	×
 toss	○	○	△	×	×
 banksalad	×	○	○	×	×
 CURVE	○	×	×	×	○

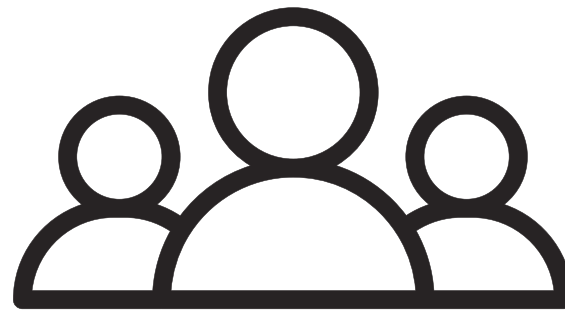
결제 후 카드 변경은 있으나,
한국에서는 법적. 결제 구조상
적용 어려움

모든 기능을 통합한 서비스의 부재 / 최적 혜택 카드 선택 서비스 부재(국내)

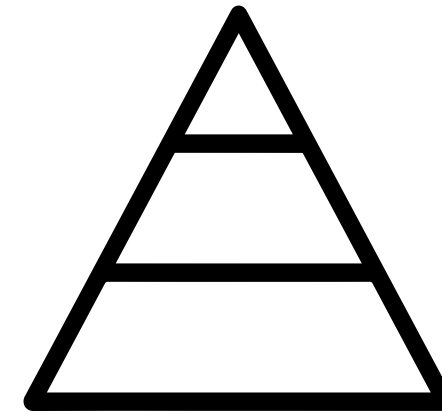
주제 선정 배경



휴면 카드의 증가



혜택 선택 피로도



피킹룰에 대한 관심 확대

- 피킹룰: 카드 사용액 대비 실제로 환급받은 혜택의 비율을 나타내는 지표

주제 선정 배경

복잡한 카드 혜택을 대신 계산해주는
AI 기반 실시간 최적 카드 추천 서비스



CardBenePICK

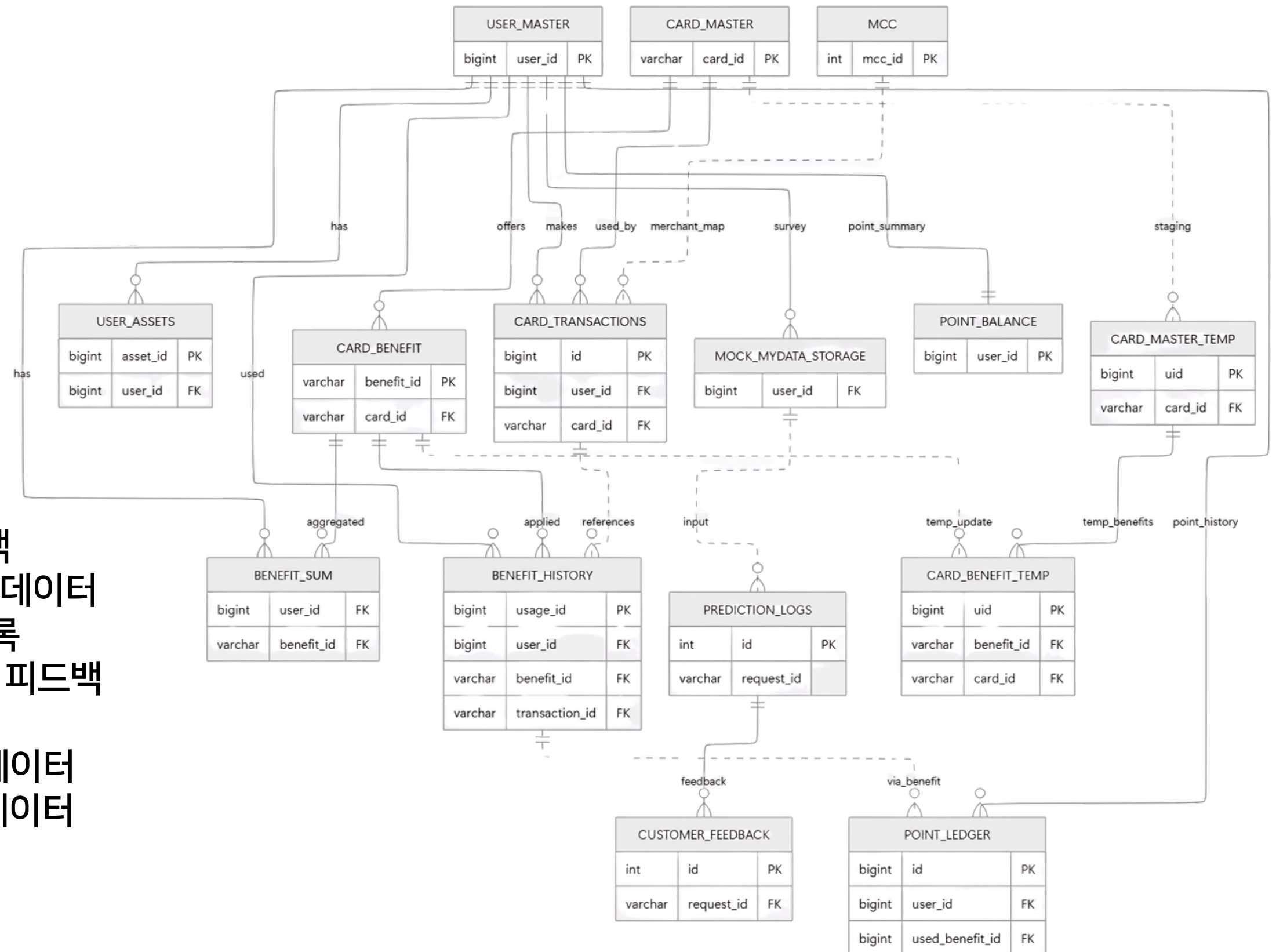
03

데이터 및 아키텍처 구성

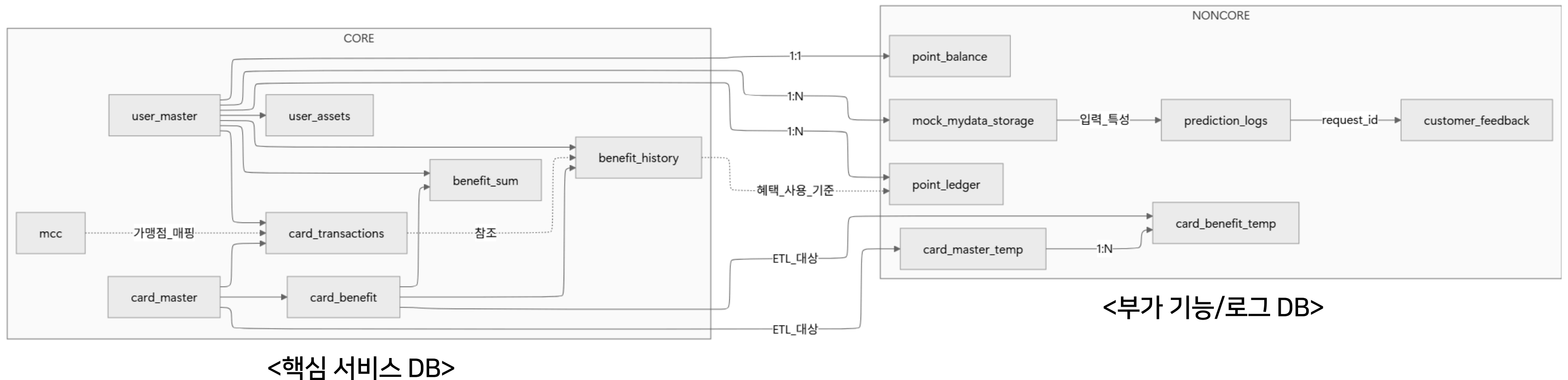
- 3-1 — 데이터 구조와 더미 데이터 생성
- 3-2 — 사용 기술 스택 및 아키텍처

(1) 데이터 구조와 더미데이터

USER_MASTER : 사용자 기본 정보
 USER_ASSETS : 보유 금융 자산 정보
 MCC : 가맹점 업종 코드
 CARD_MASTER : 카드 상품 기본 정보
 CARD_BENEFIT : 카드별 혜택 정보
 CARD_TRANSACTION : 카드 결제 내역
 BENEFIT_SUM : 사용자별 혜택 집계 요약
 BENEFIT_HISTORY : 혜택 적용 이력
 POINT_BALANCE : 사용자별 현재 포인트 잔액
 MOCK_MYDATA_STORAGE : 설문 기반 소비 패턴 데이터
 PREDICTION_LOGS : 모델 예측 입력, 출력 기록
 CUSTOMER_FEEDBACK : 설문 결과에 대한 사용자 피드백
 POINT_LEDGER : 포인트 발생, 차감 이력
 CARD_MASTER_TEMP : 카드 스펙 크롤링 원본 데이터
 CARD_BENEFIT_TEMP : 카드 혜택 크롤링 원본 데이터



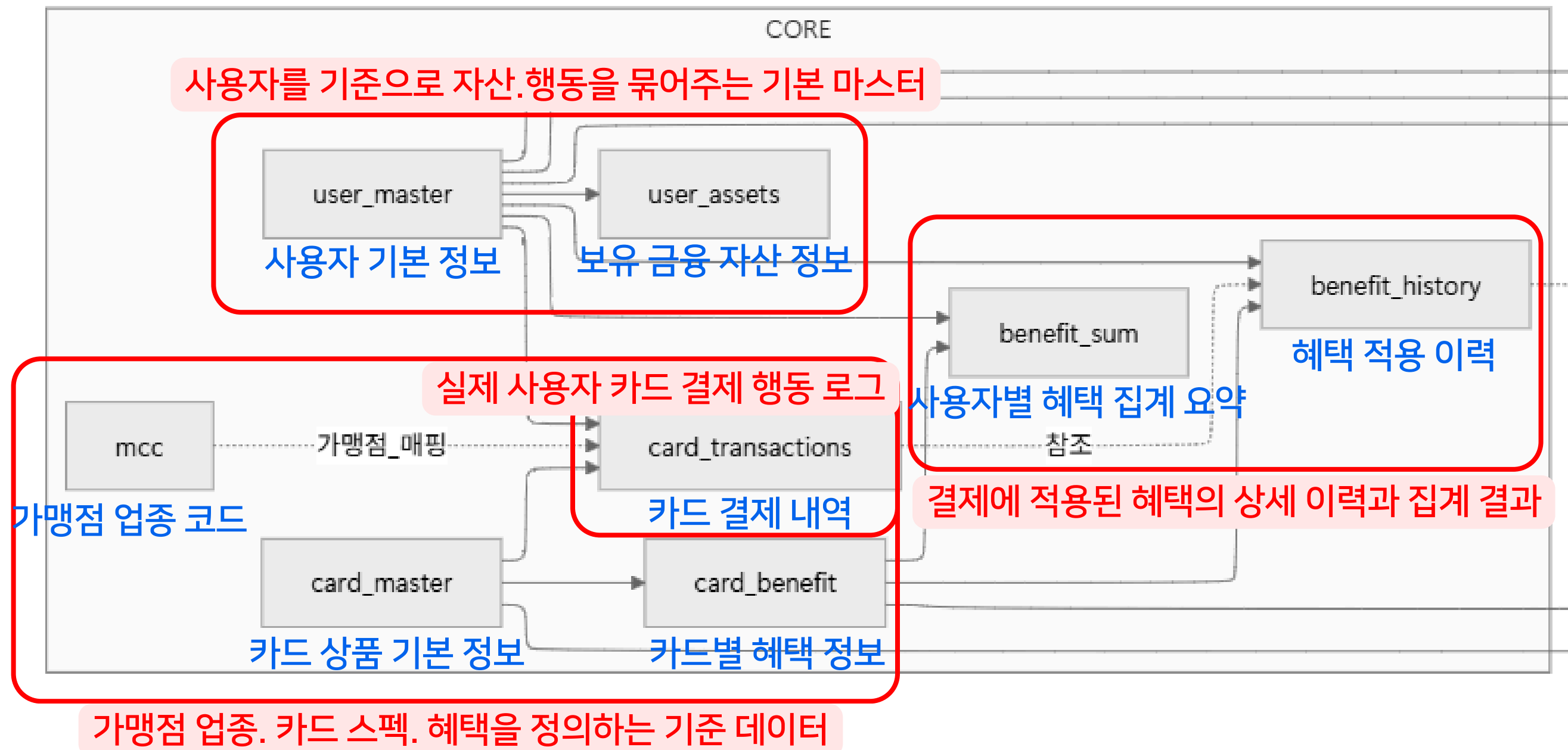
(2) DB 테이블 간 관계도



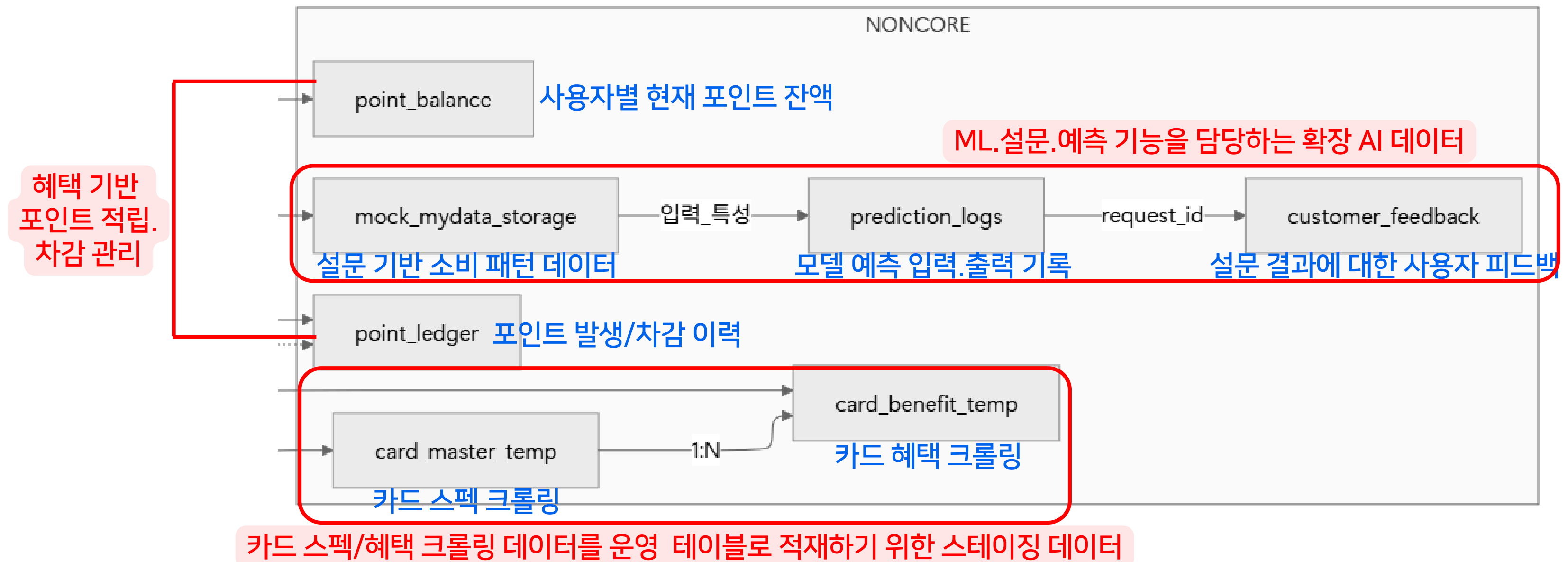
- 핵심 서비스 DB: 실제 서비스에서 발생하는 결제·사용자·카드·혜택 등의 핵심 데이터가 저장되는 테이블
- 부가 기능/로그 DB: 설문/모델/포인트/크롤링/ETL/Airflow/Log저장 등 부가 기능 또는 분석·예측을 위해 생성되는 보조 데이터 영역

(2) DB 테이블 간 관계도

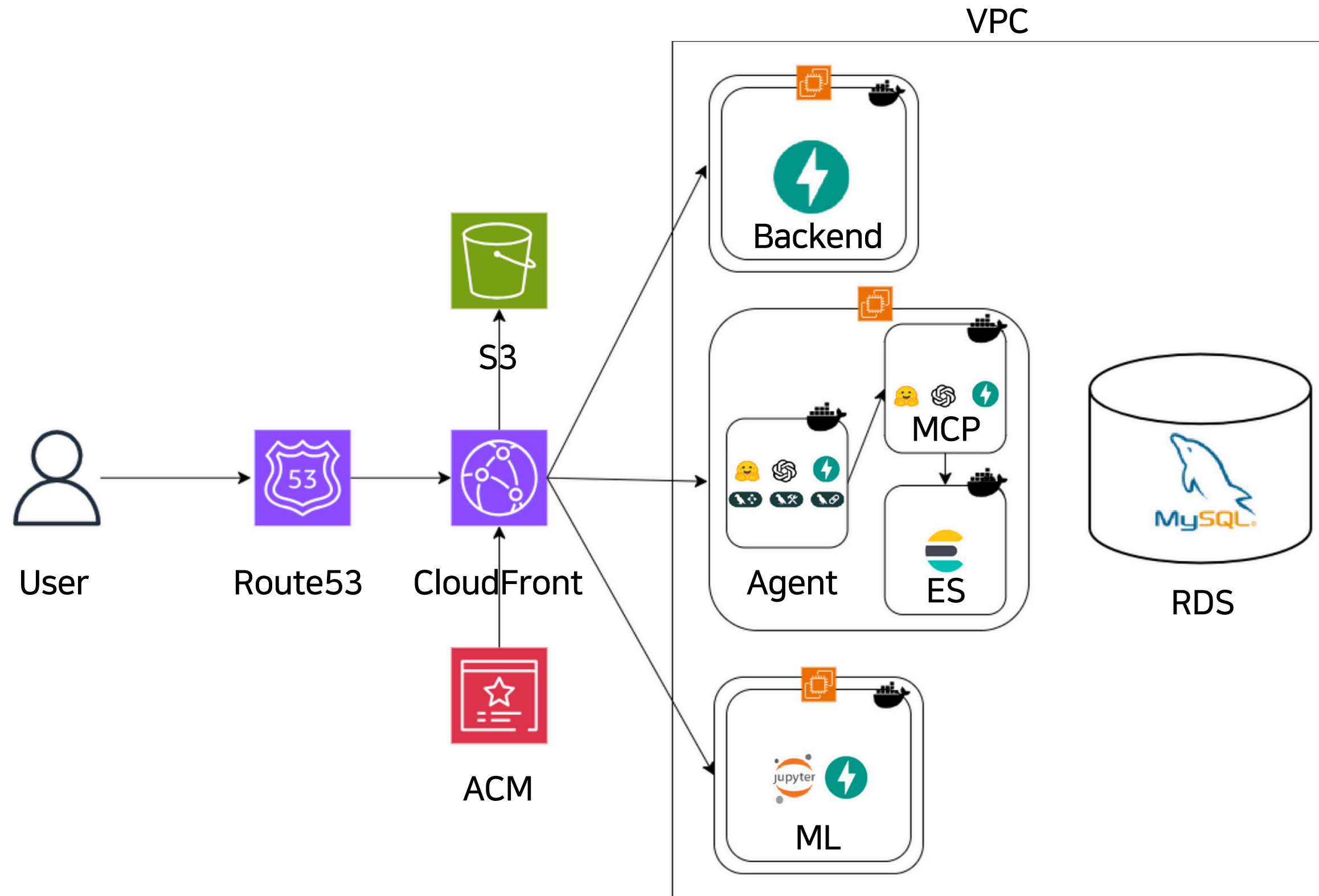
카드 고릴라 Top 100 카드의 혜택을 크롤링 후 전수 분석하여 업종, 혜택 조건, 할인 구조를 수치 기반 코드로 재정의



(2) DB 테이블 간 관계도



사용 기술 스택 및 아키텍처

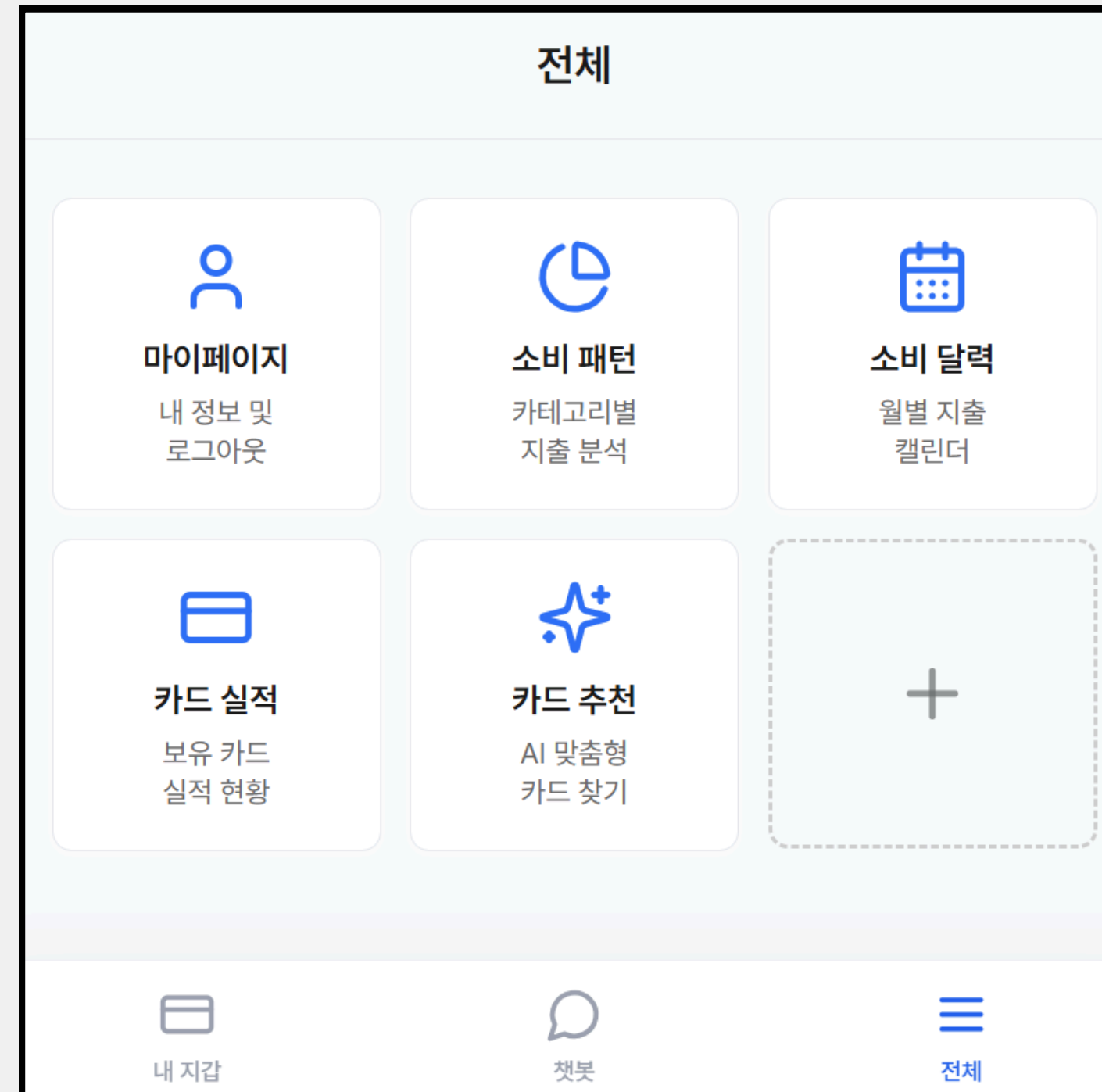


04

서비스 구성 및 구현

- 4-1 ————— 전체 서비스 기능 소개
- 4-2 ————— AI Agent 및 MCP
- 4-3 ————— RAG
- 4-4 ————— ML
- 4-5 ————— Airflow

(1) - 전체 메뉴



(2) - 마이페이지

내 정보

가입된 회원 정보입니다.

이름	홍길동
휴대폰 번호	010-1234-1234
생년월일	1990-01-01
통신사	SKT

연동된 금융사

마이데이터로 연결된 카드사 목록입니다.

신한카드 삼성카드 BC 바로카드

카드사 추가 연동하기

어떤 카드를 쓰시나요?

자주 쓰는 카드사를 선택하면
소비 패턴을 분석해 드려요.

삼성카드

삼성카드



현대카드



하나카드



롯데카드



IBK기업은행



SC카드



신한카드



KB국민카드



NH농협카드



우리카드



카카오뱅크



BC 바로카드



토스뱅크

Kbank

케이뱅크



신협카드



새마을금고

내 카드 관리

등록된 모든 카드(3장)를 관리합니다.

신한카드 Mr.Life
신한카드



삼성카드 taptap O
삼성카드



BC 바로 클리어 플러스
BC 바로카드



카드 직접 등록하기

계정 관리

[>] 로그아웃



회원 탈퇴



(3) - 소비 패턴

← 소비 분석

< 2025년 12월 >

총 지출
120,068원

↗ 소비 패턴 요약

📅 05일에 가장 많은 지출(58,403원)이 있었어요.

🕒 주로 저녁 시간대에 카드를 자주 사용하시네요.

어디에 돈을 썼을까?

● 기타 소비	60,800원
● 카페/간식	37,738원
● 식비	12,000원
● 쇼핑/마트	9,530원

📁 카드별 주요 소비처

신한카드 Mr.Life
주로 **기타 소비**에 썼어요

60,800원
(68%)

삼성카드 taptap O
주로 **카페/간식**에 썼어요

28,601원
(100%)

BC 바로 클리어 플러스
주로 **쇼핑/마트**에 썼어요

1,500원
(100%)

📋 가장 많이 쓴 카드는?

신한카드 Mr.Life

89,967원

삼성카드 taptap O

28,601원

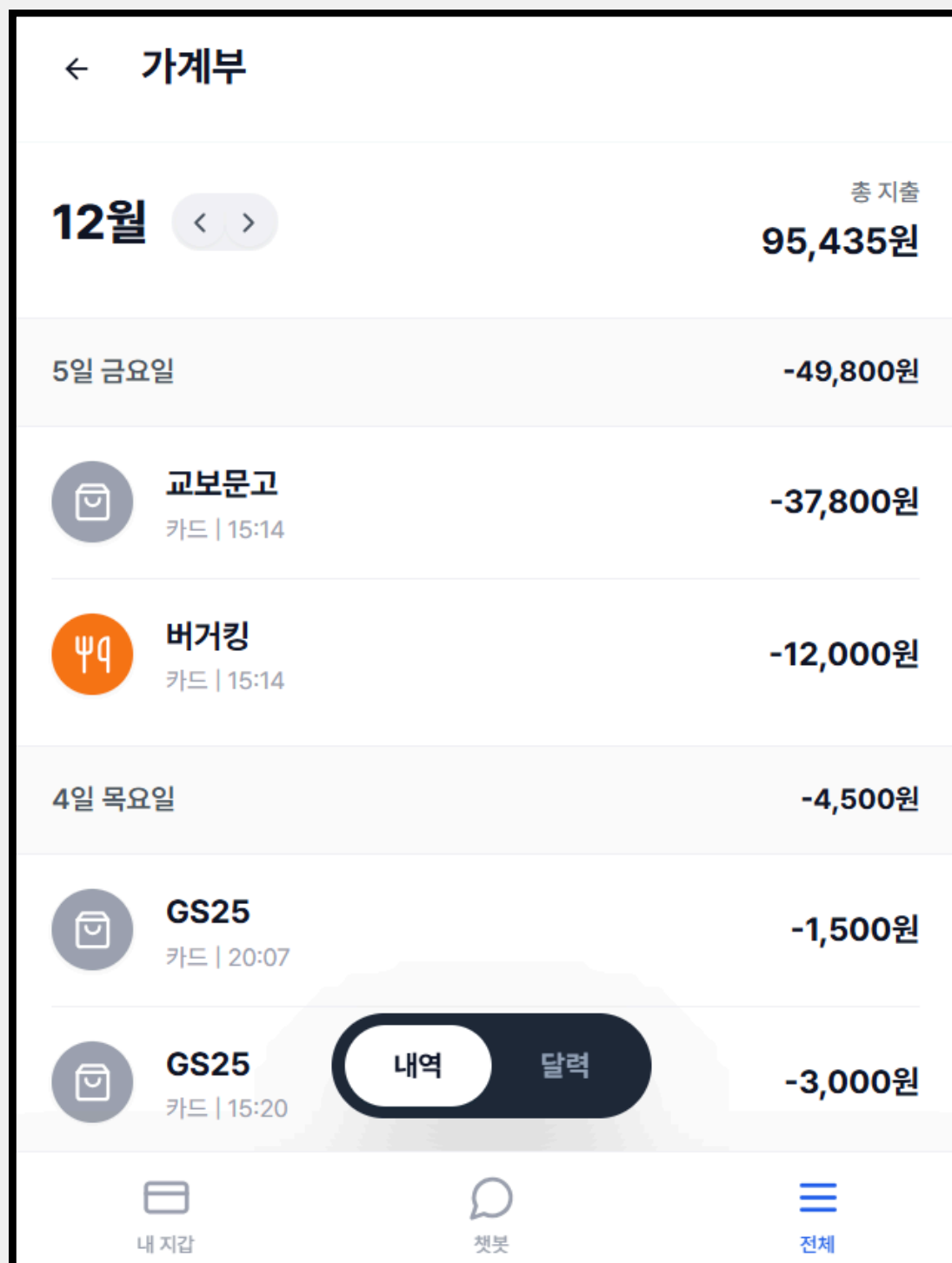
BC 바로 클리어 플러스

1,500원

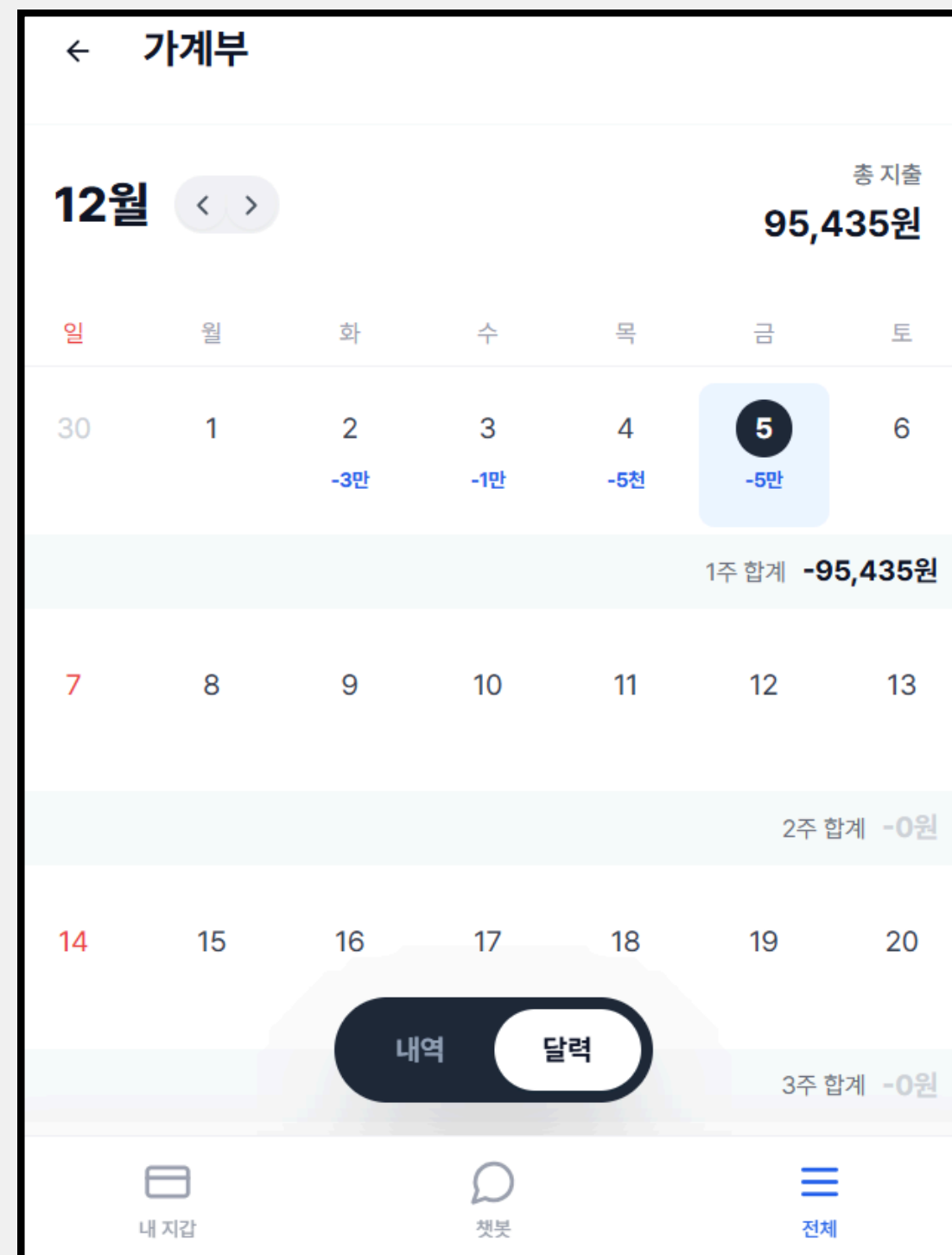
카드 실적 자세히 보기

(4) -소비 기록

소비 내역



소비 달력



(5) - 카드 실적

0

달성 완료

0

달성 임박

3

달성 필요

지난 달 실적 요약



신한카드 Mr.Life

859,290원 사용

달성



삼성카드 taptap O

398,730원 사용

달성



BC 바로 클리어 플러스

540,220원 사용

달성

이번 달 실적 현황



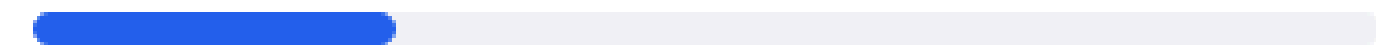
신한카드 Mr.Life

목표 300,000원

달성 필요

실적 진행률

81,937원



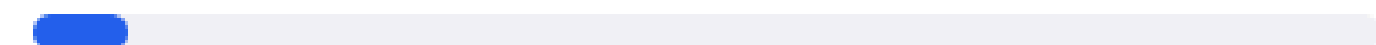
삼성카드 taptap O

목표 300,000원

달성 필요

실적 진행률

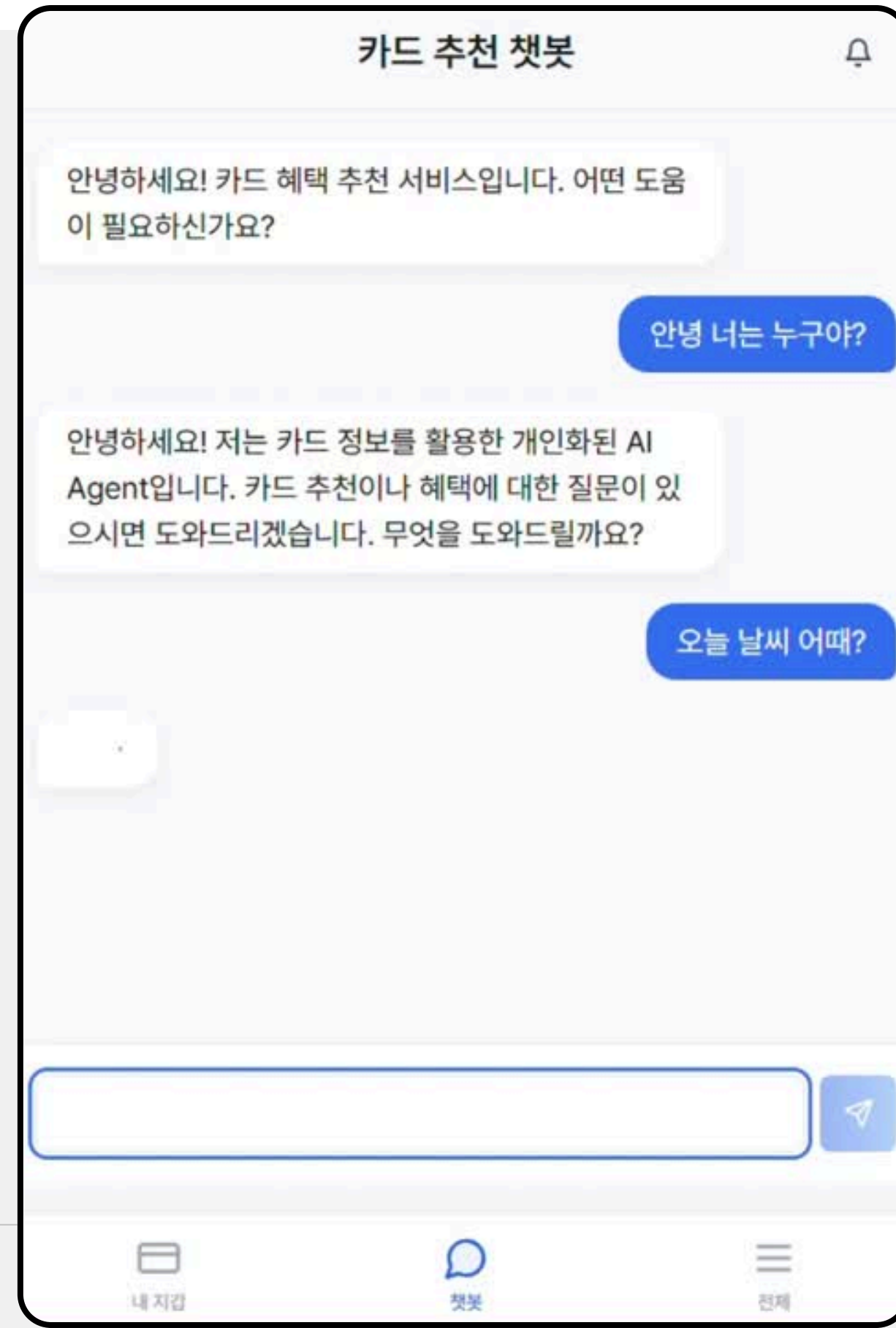
20,601원



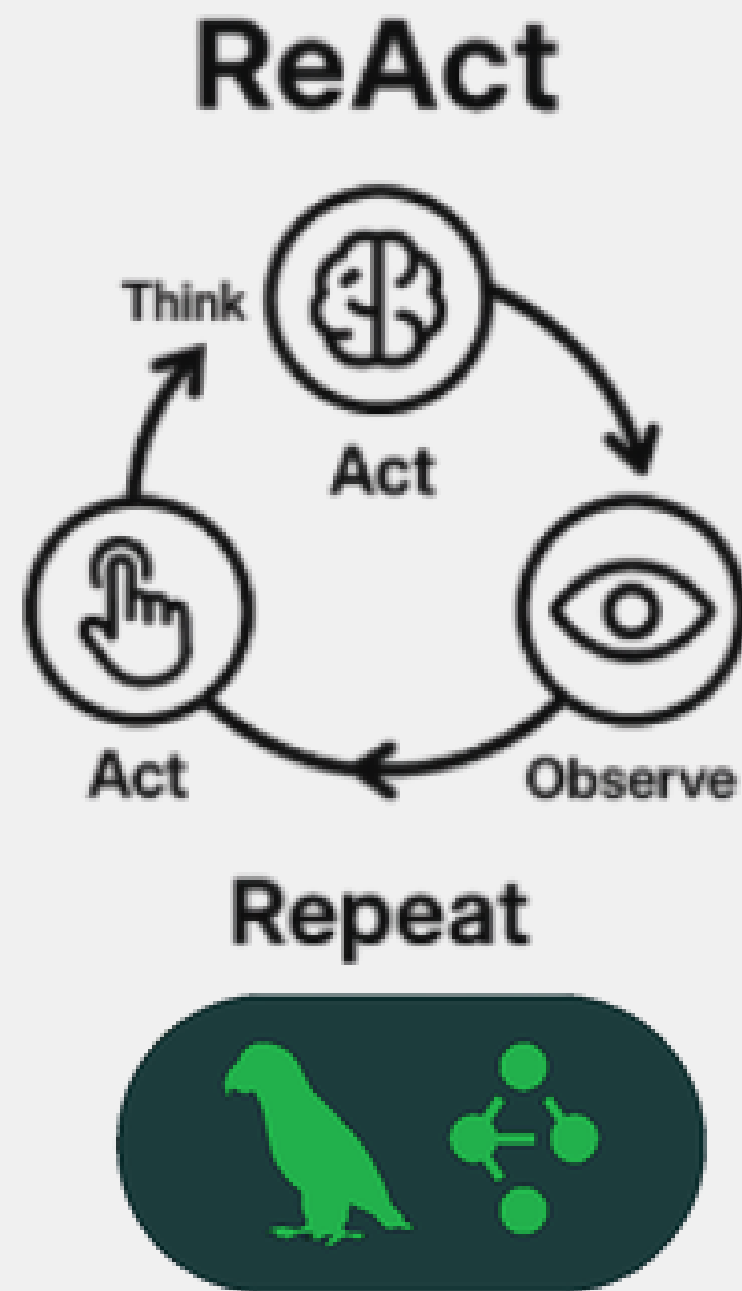
서비스 구성 및 구현 - 메인 기능 소개

AI AGENT

AI Agent 시연 영상



(1) AI Agent



LangGraph ReAct Agent

```

SYSTEM_PROMPT = (
  "당신은 카드 정보를 활용한 개인화된 AI Agent 입니다..\n"
  "\n"
  "절대 원칙:\n"
  "- 근거 없는 추측이나 임의의 지식 생성(할루시네이션)을 절대 하지 않습니다.\n"
  "- MCP 서버의 도구 결과에 반드시 근거해야 합니다.\n"
  "- MCP 도구를 사용한다면 어떤 도구(함수 이름)를 사용했는지도 표현합니다.\n"
  "- 한국어로 대답합니다."
  "- 도구 결과나 문서에서 확인할 수 없는 정보는 반드시 '모르겠습니다'고 명시합니다.\n"
  "\n"
  "행동 지침:\n"
  "1. MCP 서버에 등록된 MCP 도구를 이용해 답변을 해야 합니다.\n"
  "\n"
  "추가 원칙:\n"
  "- 자신의 내부 지식을 근거로 확장하거나 추측하지 않습니다.\n"
  "- 문서나 도구 결과가 모호할 경우, 모호하다고 명시합니다.\n"
  "- 답변을 할 때 답변 이유가 존재하면 보기 좋게 이유를 정리해서 답변해야 합니다.\n"
  "- 적절한 개행 문자를 사용해서 답변을 보기 좋게 구분해야 합니다.\n"
  "- 반드시 사실 기반의 답변만 제공합니다.\n"
)
  
```

model : gpt-4o-mini / System Prompt

(1) AI Agent

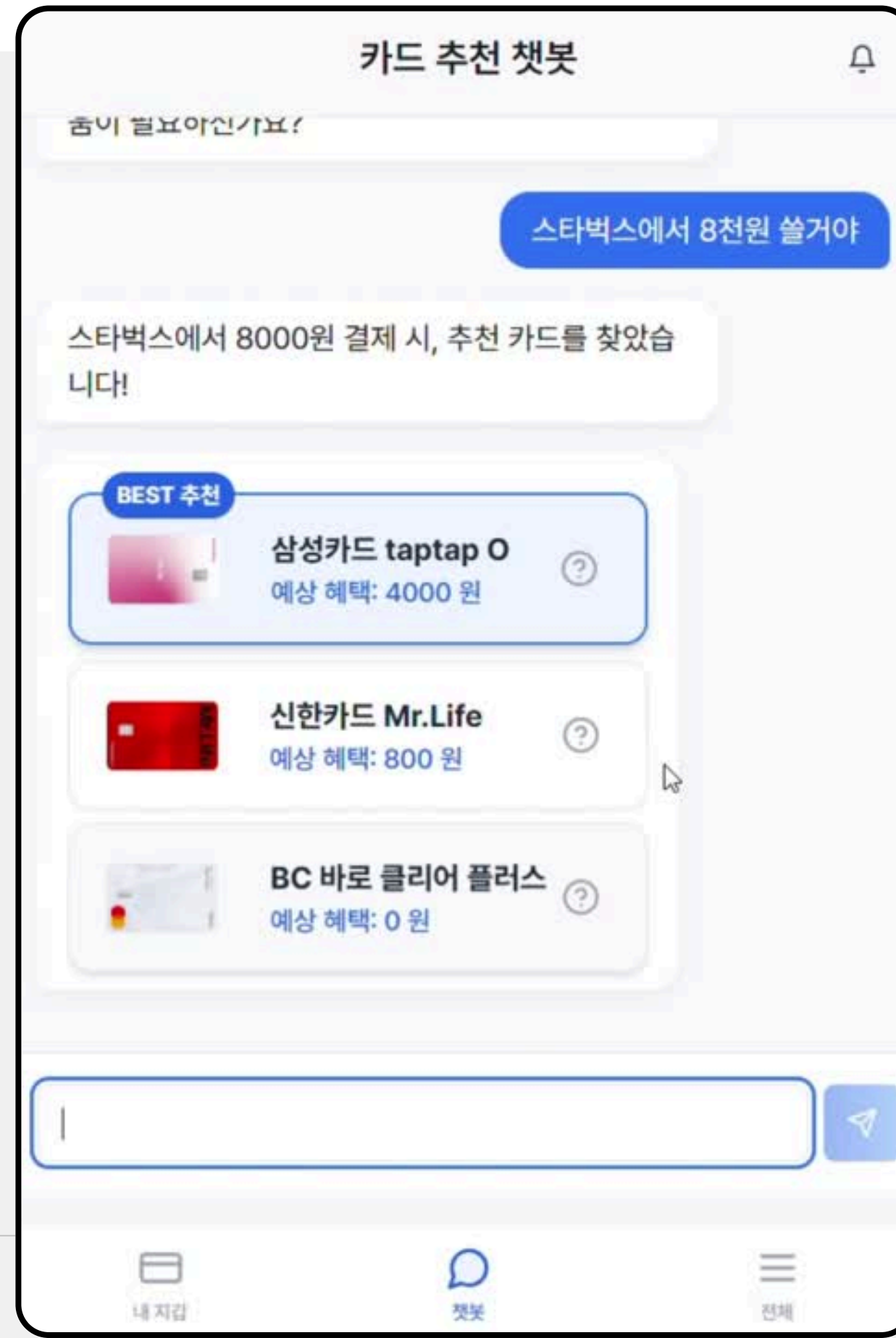
```
def _get_session(session_key: SessionKey) -> Dict[str, Any]:  
    if session_key not in user_sessions:  
        user_sessions[session_key] = {  
            "history": [],  
            "auth": None,  
            "user_id": session_key if isinstance(session_key, int) else None  
        }  
    return user_sessions[session_key]
```

```
def get_history(session_key: SessionKey = DEFAULT_SESSION_KEY) -> List[BaseMessage]:  
    return _get_session(session_key)["history"]
```

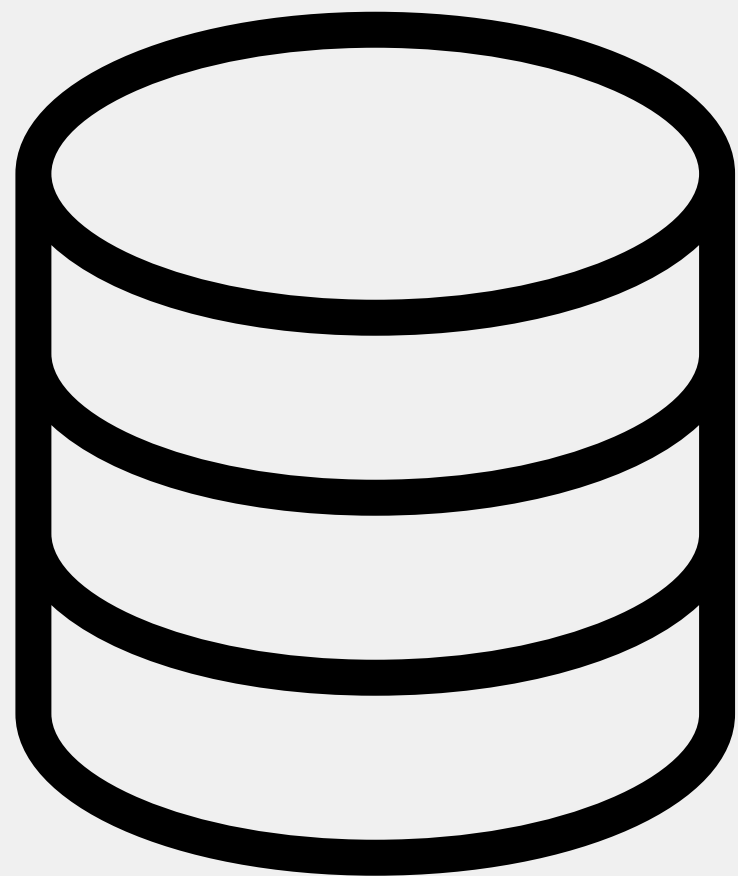
멀티턴 구성

Session Key / 전역 저장소 활용 history 유지

MCP 시연 영상



(2) 챗봇 기능 AI Agent 및 MCP



가맹점 MCC 코드

사용자 보유 카드

보유 카드별 혜택

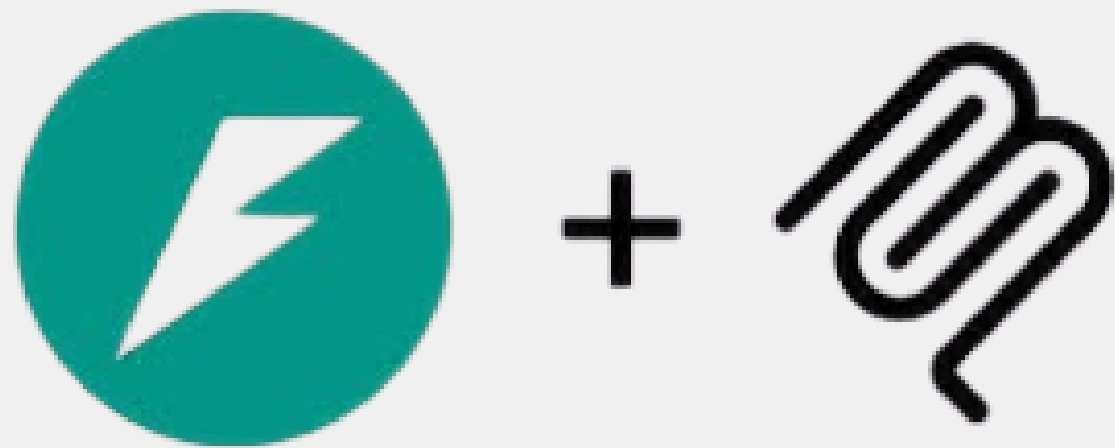
혜택별 사용 한도

당월 사용 혜택

전월 실적 충족 여부

데이터베이스와 사용자 마이데이터에서 가져와야 할 정보

(2) 챗봇 기능 AI Agent 및 MCP



```
client = MultiServerMCPClient({  
    "fisa-mcp": {  
        "url": "http://mcp_server_final:8001/mcp",  
        "transport": "streamable_http"  
    }  
})
```

MCP Tool : FastAPIMCP



가맹점 MCC 코드

사용자 보유 카드

보유 카드별 혜택

혜택별 사용 한도

당월 사용 혜택

전월 실적 충족 여부

각각의 기능을 Tool로 구성

(2) 챗봇 기능 AI Agent 및 MCP

asset_id	user_id	asset_type	institution_name	external_account_id	external_account_name	currency	balance	updated_at	created_at
1	1	card	신한카드	13	신한카드 Mr.Life	KRW	0.00	2025-12-02 18:34:35	2025-12-02 09:30:22
2	1	card	삼성카드	51	삼성카드 taptap O	KRW	0.00	2025-12-02 18:34:39	2025-12-02 09:31:05
3	1	card	BC 바로카드	2346	BC 바로 클리어 플러스	KRW	0.00	2025-12-02 18:34:42	2025-12-02 09:31:29
12	11	card	현대카드	2753	the Green Edition3	KRW	0.00	2025-11-27 03:02:12	2025-11-27 03:02:12
18	12	card	롯데카드	2330	LOCA 365 카드	KRW	0.00	2025-12-01 00:54:27	2025-12-01 00:54:27
19	12	card	KB국민카드	2609	쿠팡 와우카드	KRW	0.00	2025-12-01 00:55:12	2025-12-01 00:55:12

benefit_id	card_id	category	summary	json_rawdata	mcc_code
115_0	115	주유	주유소, 충전소 업종 리터당 60원 청구할인	[{"subtitle": "주유소, 충전소 업종 리터당 60원 ...	["3106", "3107"]
115_1	115	대형마트	대형마트 10% 청구할인	[{"subtitle": "대형마트 10% 청구할인", "descripti...	["8341", "8343", "8344"]
115_2	115	통신	전화요금, 인터넷이용료, 케이블TV 업종 10% ...	[{"subtitle": "전화요금, 인터넷이용료, 케이블TV ...	["1111", "1112", "1113", "1114", "1115", "1116.
115_3	115	교통	버스, 지하철, 택시 업종 10% 청구할인	[{"subtitle": "버스, 지하철, 택시 업종 10% 청구...	["3011", "3012", "3014", "3021"]
115_4	115	해외	해외이용(해외직구) 5% 할인 캐시백	[{"subtitle": "해외이용(해외직구) 5% 할인 캐시...	[]

user_id	benefit_id	day_amount	day_count	week_amount	week_count	month_amount	month_count	year_amount	year_count
1	13_0	0	0	3000	1	3000	1	3000	1
1	13_1	0	0	1200	3	1200	3	1920	4
1	13_2	0	0	0	0	0	0	0	0
1	13_6	340	1	1140	2	1140	2	6012	16
1	2346_1	0	0	1067	2	1067	2	1067	2
1	2346_2	0	0	440	3	440	3	440	3

개별 Tool이 다양한 DB 값을 조회하여 리턴

(2) 챗봇 기능 AI Agent 및 MCP

```
@app.get("/sale", operation_id="get_sale_value")
def get_sale(user_id :int, merchant: str, mcc_code : int, benefits : list, limits : list, amount: int = None)
    """
    가맹점 이름과 결제금액, 사용자 보유 카드 혜택을 이용하여 가장 결제 금액이 저렴한 카드와 결제 정보를 반환합니다.

    이 함수를 실행하기 전 필수 정보 수집 과정:
    1. 사용자의 정보가 필요하면 get_user_id() 도구를 사용하세요
    2. merchant의 MCC 코드가 필요하면 get_mcc_code() 도구를 사용하세요
    3. 해당 가맹점의 사용자 보유 카드 혜택 정보를 조회하려면 get_benefits_by_mcc() 도구를 사용하세요
    4. 사용자의 혜택 적용 한도 정보를 조회하려면 get_user_benefit_limit() 도구를 사용하세요
    5. 모든 정보가 수집되면 이 함수를 호출하여 최종 카드를 추천받으세요
    """
```

```
\\"recommended_card\\": {
  \\"card_id\\": 1,
  \\"card\\": \\"신한 Mr.Life\\",
  \\"merchant_name\\": \\"GS25\\"
  \\"benefit_id\\" : 51_1,
  \\"price\\": 12000,
  \\"bene_val\\": 1200,
  \\"final_val\\": 10800,
  \\"reason\\": \\"신한 Mr.Life:
}}}
```

Tool 순차 사용을 통한 최종 혜택 계산

JSON 형태의 출력 예시 Prompt

model : openai/gpt-oss-120b

(2) 챗봇 기능 AI Agent 및 MCP

카드 추천 요청 정보

사용자 ID: 1

가맹점: 스타벅스

MCC 코드: 4041

결제 예정 금액: 3,900원 (예상)

결제 시각: 2025년 12월 07일 21시 46분 01초 (일요일)

BENEFIT #1

card_name: 신한카드 Mr.Life

benefit_id: 13_6

card_id: 13

category: 푸드

summary: 식음료 10% 할인

json_rawdata:

■ 오후 9시 ~ 오전 9시까지 Night 할인서비스 10% 할인

일간 혜택 적용 금액: 600원

일간 혜택 적용 횟수: 1

주간 혜택 적용 금액: 1,740원

주간 혜택 적용 횟수: 3

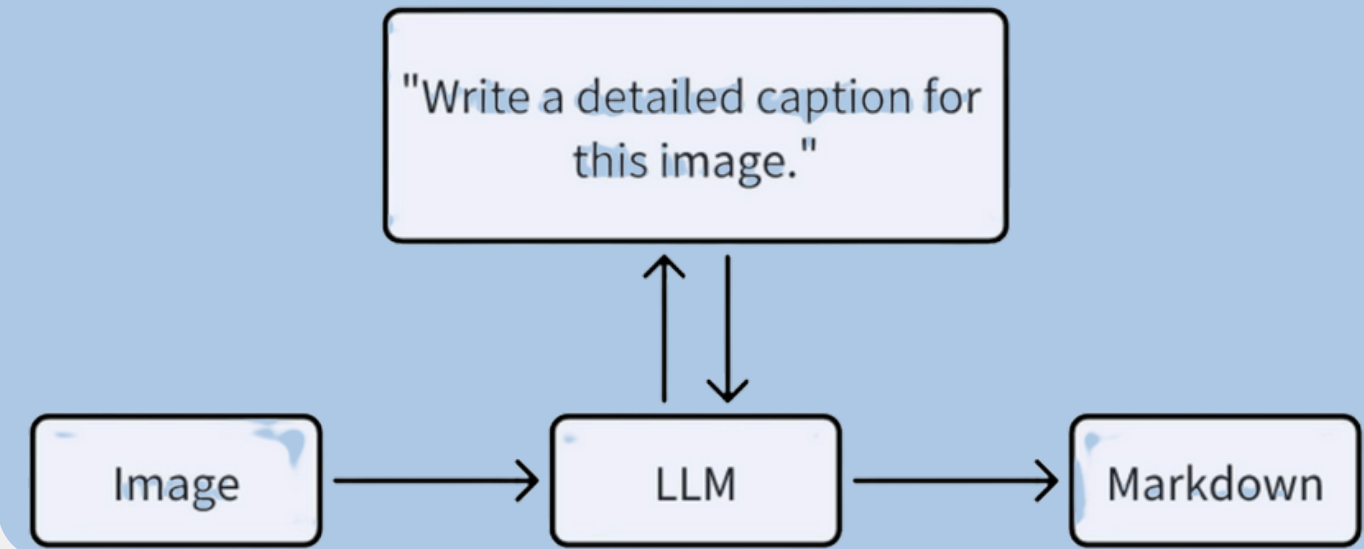
월간 혜택 적용 금액: 1,740원

월간 혜택 적용 횟수: 3

연간 혜택 적용 금액: 6,612원

연간 혜택 적용 횟수: 17

전월 카드 사용 실적: 859,290원



[표 데이터]

구분(택1) | 커피 | 쇼핑

- 패키지1 | 스타벅스 50% 할인 | 오픈마켓 7% 할인 | 소셜커머스 1% 적립 | 트렌디숍 1% 적립
- 패키지2 | 스타벅스 50% 할인 | 소셜커머스 7% 할인 | 오픈마켓 1% 적립 | 트렌디숍 1% 적립
- 패키지3 | 스타벅스 50% 할인 | 트렌디숍 7% 할인 | 오픈마켓 1% 적립 | 소셜커머스 1% 적립
- 패키지4 | 커피전문점 30% 할인 | 오픈마켓 7% 할인 | 소셜커머스 1% 적립 | 트렌디숍 1% 적립
- 패키지5 | 커피전문점 30% 할인 | 소셜커머스 7% 할인 | 오픈마켓 1% 적립 | 트렌디숍 1% 적립
- 패키지6 | 커피전문점 30% 할인 | 트렌디숍 7% 할인 | 오픈마켓 1% 적립 | 소셜커머스 1% 적립

카드 혜택별로 분리 후 마크다운으로 재구조화

성능 평가 - AI Agent/MCP 카드 혜택 계산 성능 평가

1. openai/gpt-oss-120b

- 정확도 : 80 %
- 응답속도 : 평균 5.3초

2. gpt-4o-mini

- 정확도 : 38%
- 응답속도 : 평균 27.1 초

3. Qwen/Qwen3-8B

- 정확도 : 56%
- 응답속도 : 평균 18.2 초

정확도	80%	38%	56%				
Test	openai/gpt-oss-120b	gpt-4o-mini	Qwen/Qwen3-8B	Test			
1	동일	상이	동일	26	동일	상이	상이
2	동일	상이	동일	27	동일	상이	상이
3	동일	상이	동일	28	동일	상이	상이
4	동일	상이	상이	29	동일	상이	상이
5	동일	상이	상이	30	동일	상이	상이
6	동일	상이	상이	31	동일	동일	동일
7	동일	상이	동일	32	동일	동일	동일
8	동일	상이	동일	33	동일	동일	동일
9	동일	상이	동일	34	동일	동일	동일
10	상이	상이	상이	35	동일	동일	동일
11	상이	상이	상이	36	동일	동일	동일
12	상이	상이	상이	37	동일	동일	동일
13	상이	동일	동일	38	동일	동일	동일
14	동일	동일	동일	39	동일	동일	동일
15	동일	동일	동일	40	동일	동일	동일
16	동일	상이	동일	41	동일	동일	동일
17	동일	상이	동일	42	동일	상이	상이
18	동일	상이	동일	43	동일	동일	동일
19	상이	상이	상이	44	동일	동일	동일
20	상이	상이	상이	45	동일	동일	동일
21	상이	상이	상이	46	동일	동일	동일
22	상이	상이	상이	47	동일	동일	동일
23	상이	상이	상이	48	동일	상이	상이
24	상이	상이	상이	49	동일	상이	상이
25	동일	상이	상이	50	동일	상이	상이

각 모델별 정확도

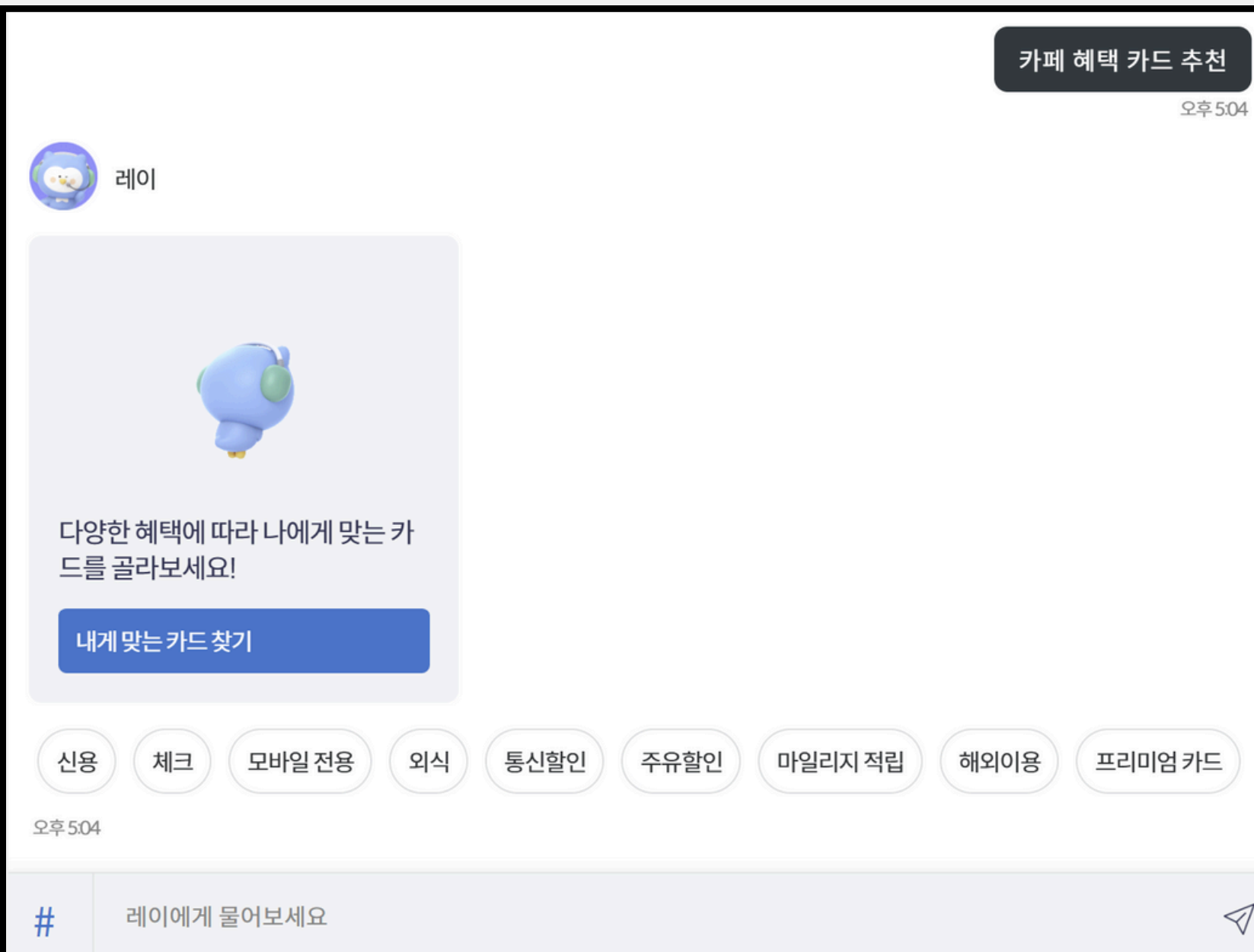
서비스 구성 및 구현 -메인 기능 소개

RAG

RAG - 기존 카드사 챗봇의 카드 추천 서비스

카드사 챗봇 예시 1)

자연어 입력: 카페 혜택 카드 추천



시나리오 버튼 외식 클릭 시 카드 추천



RAG - 기존 카드사 챗봇의 카드 추천 서비스

카드사 챗봇 예시 2)

자연어 입력: 카드사에서 인기있는 카드 추천

카페 혜택 카드 추천'

오후 7:37

말씀하신 카드가 없습니다. 삼성카드가 추천하는 카드를 보여드릴게요.

인기 있는 삼성카드를 추천해드릴게요.



삼성 ID ON 카드

[국내] 본인 20,000원 / 가족 13,000원

[MASTER] 본인 20,000원 / 가족



삼성카드 taptap O

[국내] 본인 10,000원

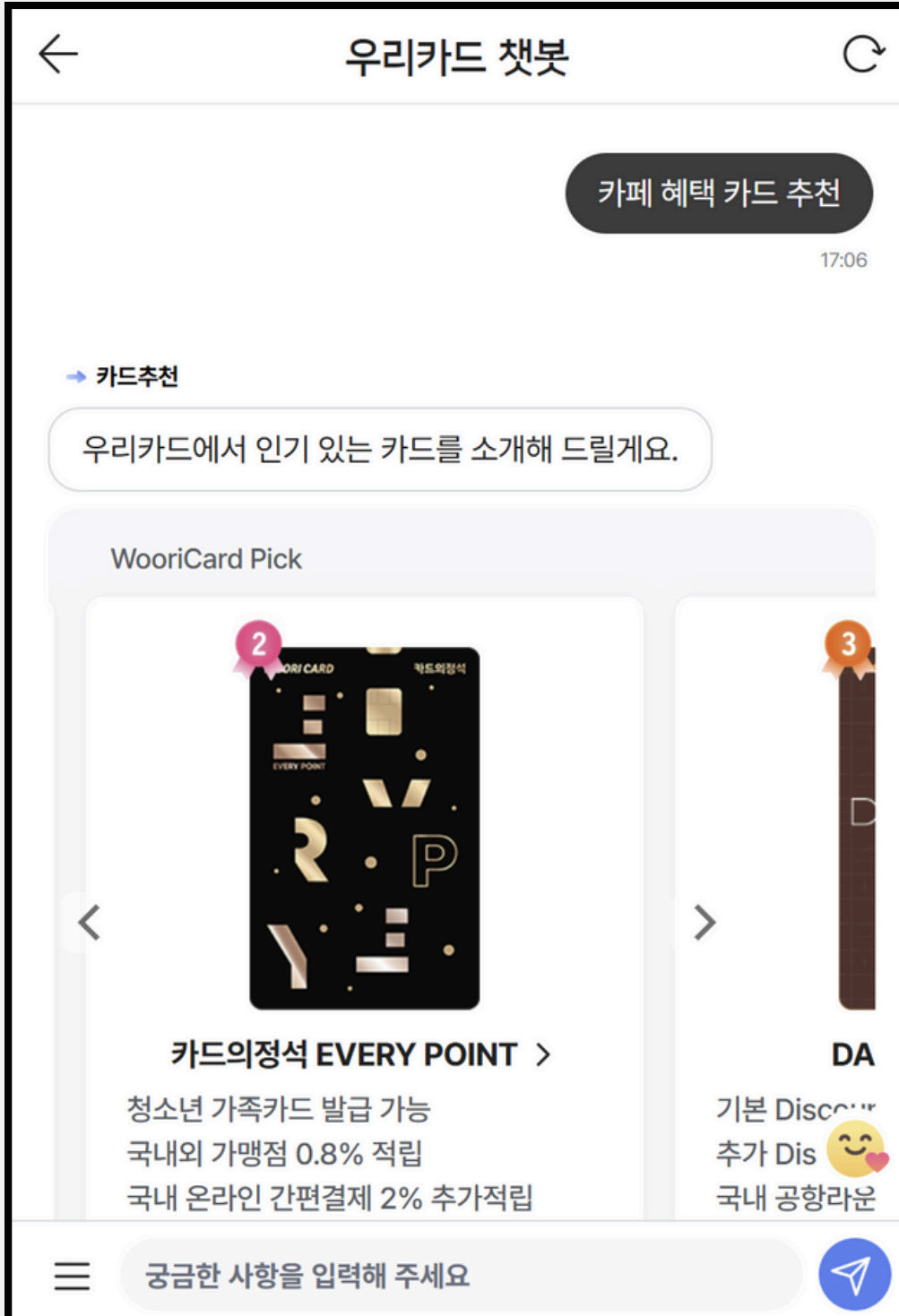
[MASTER] 본인 10,000원



삼성카드 & MILEAGE PLATINUM (스카이패스)

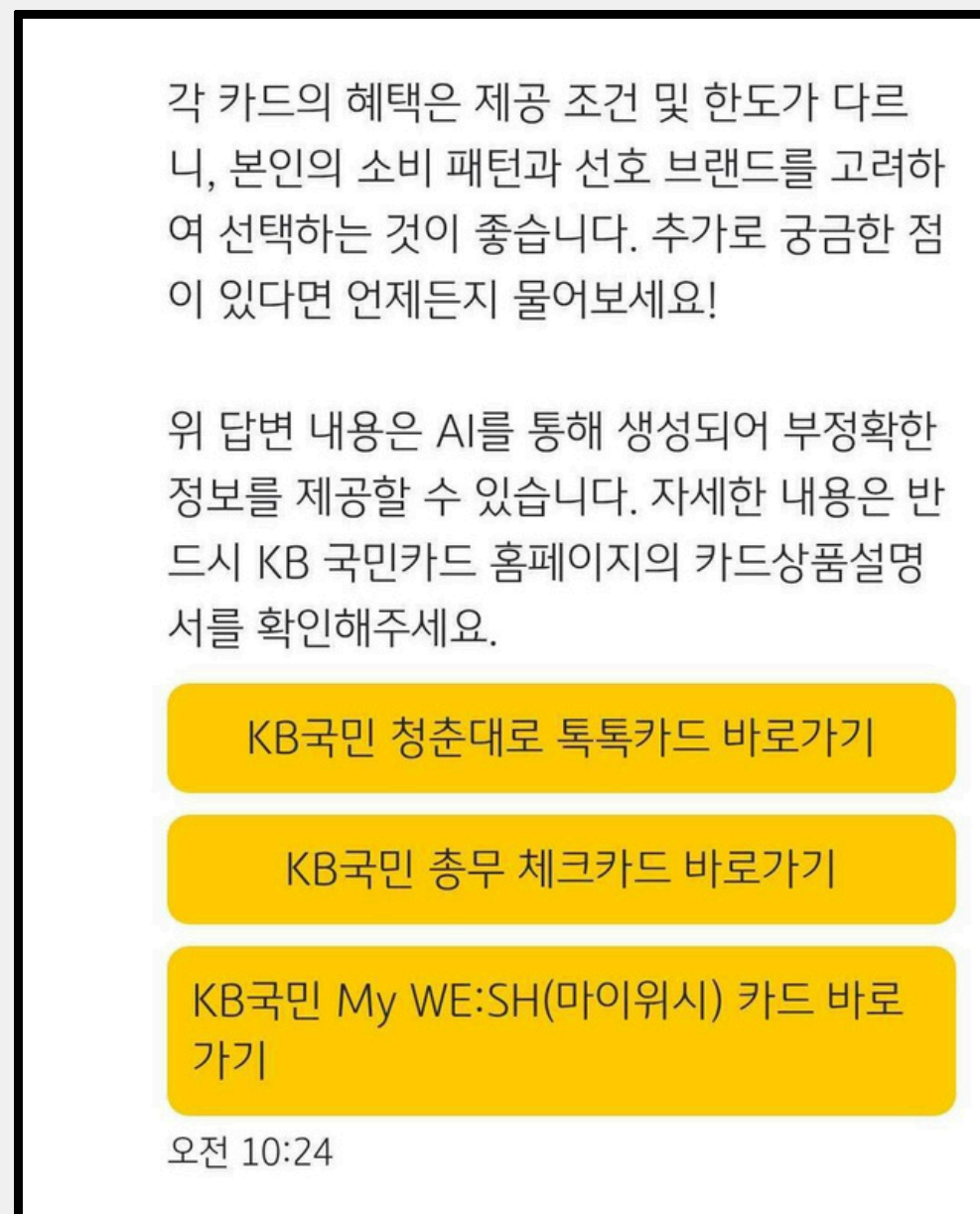
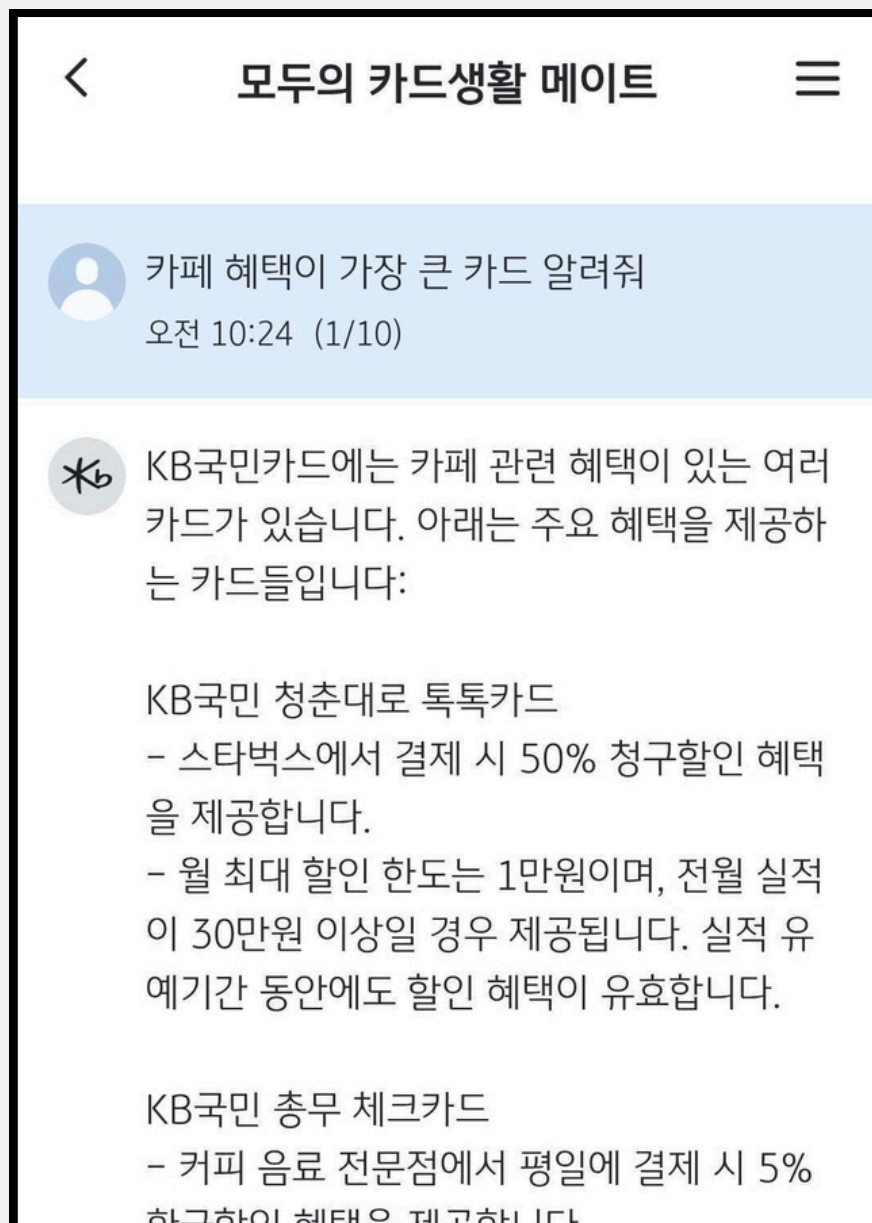
[국내] 본인 47,000원

[AMEX] 본인 49,000원

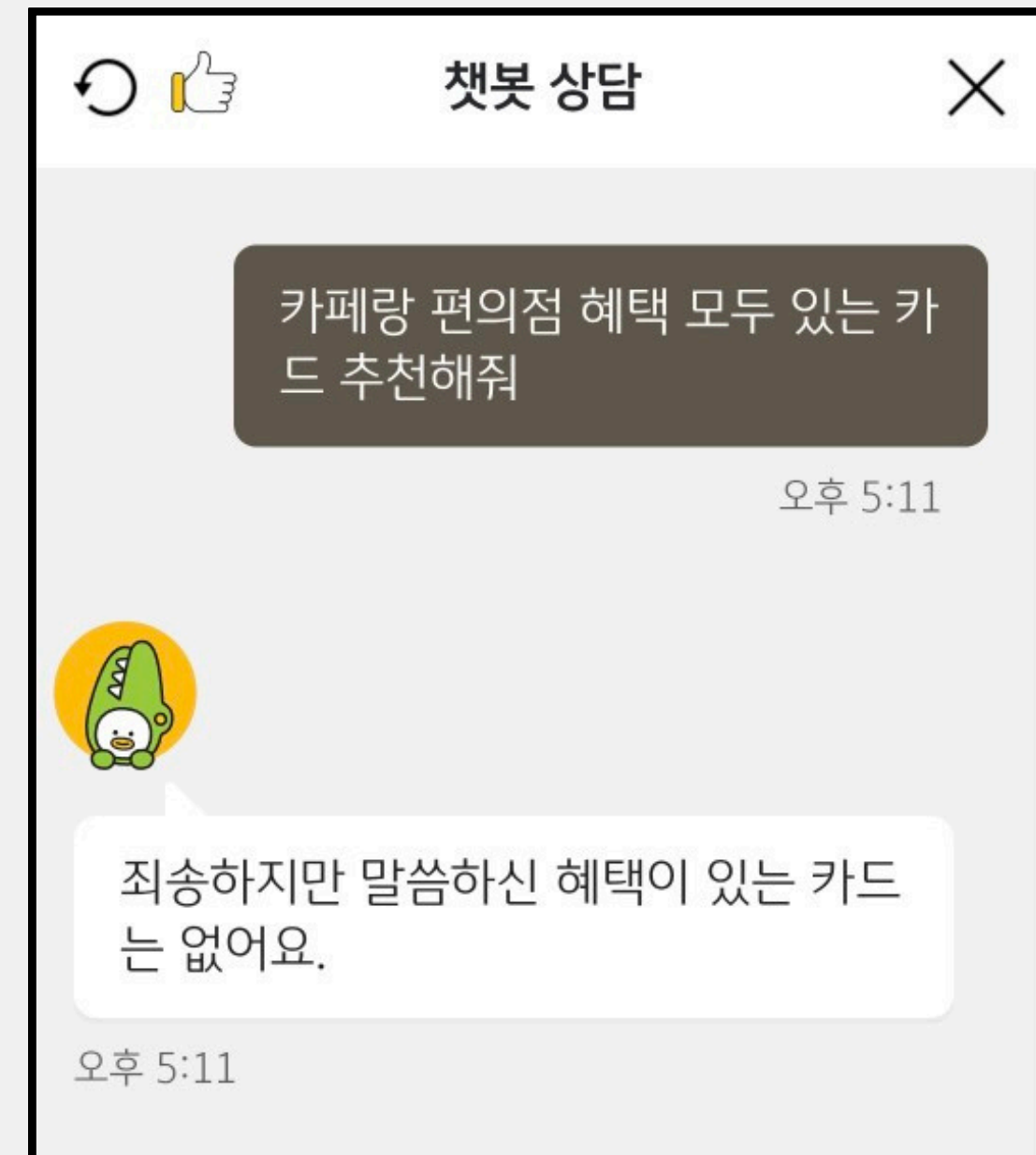


RAG - 기존 카드사 챗봇의 카드 추천 서비스

카드사 챗봇 예시 3) 단일 혜택 검색: 검색 가능



복합 혜택: 해당 카드사에 두 혜택을 모두 가진 카드가 존재하지만 검색 불가



RAG - 목표 정의

⚠ 단일 검색의 한계

현재 챗봇은 “카페 할인 카드”와 같이 단일 혜택 키워드 기반 질문에는 정확하게 반응할 수 있지만
“카페랑 편의점 혜택 모두 있는 카드 추천해줘”와 같은
여러 혜택을 동시에 충족하는 카드는 찾지 못하는 경우가 많다.



📌 목표: 복합 의도 파악 기반 카드 추천 솔루션

복잡한 혜택을 알아듣고 최대한 요구받은 혜택 중 많은 혜택을 가진 카드를 추천

RAG - 데이터 구조화 (Elasticsearch Nested Type)

Parent 영역: 카드의 메타데이터 및 카드 기본데이터

Nested 영역: 혜택 하나하나를 독립된 문서로 취급하여
개별 임베딩 벡터 생성.

Why Elasticsearch?

"BM25(키워드)와 Vector(의미) 검색을 단일 쿼리로
통합하는 강력한 Hybrid Search"

"독립된 혜택 객체들의 매칭 점수를
정교하게 합산(Sum)하는 Nested Query 지원"

```
{
  // [Parent Document] 카드 기본 정보 (Global Context)
  "card_id": "13",
  "card_name": "신한카드 Mr.Life",
  "card_company": "신한카드",
  "card_type": 0,
  "domestic_year_cost": 15000, // 연회비
  "previous_month_performance": 300000, // 전월 실적
  "card_link": "https://www.card-gorilla.com/...",

  // [Nested Documents] 독립적인 혜택 객체의 리스트
  "benefits": [
    {
      "category": "공과금",
      "summary": "월납요금(공과금) 10% 할인서비스",
      "subtitles": [
        {
          "subtitle": "대상 상세",
          "descriptions": ["전기요금", "도시가스", "통신요금(SKT, LG U+, KT)"]
        }
      ],
      "vector": [0.013, -0.24, 0.55, ... , 0.12]
    },
    {
      "category": "편의점",
      "summary": "편의점 10% 할인",
      "subtitles": [ ... ],
      "vector": [-0.15, 0.33, 0.02, ... , -0.88]
    },
    {
      "category": "병원/약국",
      "summary": "병원/약국업종 10% 할인",
      "subtitles": [ ... ],
      "vector": [0.71, -0.11, 0.45, ... , 0.09]
    },
    // ... 나머지 혜택들 ...
  ]
}
```

Parent
영역

Nested
영역

모델 선정 (Embedding & LLM)

Embedding Model: [BAAI/bge-m3](#)

Why?

한국어 문맥(조사/어미) 처리 최적화

최대 8192 토큰의 긴 입력(Long Context) 지원 (→ 긴 혜택 설명도 잘림 없이 처리)

LLM Model: [Meta Llama-3.1-8B-Instruct](#)

Why?

Instruction Following(지시 이행) 능력 탁월

JSON 포맷 준수율 높음.

한국어와 맥락 이해능력 출중:

Gemma-2-9b 모델은 한국어 쿼리 내 혜택 키워드 식별 실패 (Context 이해 부족)

RAG 전략 - 쿼리 최적화: 그룹핑 및 동적 가중치

[Issue 1] 키워드 중복으로 인한 점수 편향

Query: "카페 중에서도 스타벅스를 자주 가는데
이거랑 주유 혜택 추천해줘"

단순 키워드 추출: [Cafe, Starbucks, Gas]

연관 키워드를 하나의 의도 그룹으로 묶어
1:1 가중치 밸런싱.

그룹핑:

```
"groups": [
  {"name": "Cafe", "keywords": ["Starbucks"]},
  {"name": "Gas", "keywords": ["Gas Station"]}
]
```

[Issue 2] 필수 조건과 선호 조건의 구분

Query: "카페, 주유는 꼭(Must) 필요하고
간편결제도 있으면 좋겠어"

단순 키워드 추출: [Cafe, Starbucks, Gas]



```
{ "is_must": true, "keywords": ["카페"],
  "name": "Cafe", "search_query": "카페 할인",
  "weight": 1 },
{ "is_must": true, "keywords": ["주유"],
  "name": "Gas", "search_query": "주유소 리터당 할인",
  "weight": 1 },
{ "is_must": false, "keywords": ["간편결제"],
  "name": "Payment", "search_query": "간편결제 혜택",
  "weight": 0.8}
```

RAG 전략 - 추천 카드 선정: 검색 및 리랭킹 고도화 전략

[1] Hybrid Search

키워드 매칭(BM25)과
의미 기반 검색(Vector/kNN)을
동시 수행.



"스타벅스" 같은 고유명사의 정확성과
"커피 할인" 같은 문맥적 의미를
모두 놓치지 않기 위함.

[2] RRF Merge (Reciprocal Rank Fusion)

서로 다른 스코어 체계를 가진
두 검색 결과(키워드 vs 벡터)를
'순위(Rank)' 기반으로 통합.



키워드 점수와 벡터 유사도 점수의
스케일 차이를 보정하여 공정하고
균형 잡힌 후보군을 도출.

[3] Advanced Reranking (Cross-Check)

1차 검색된 후보군을 대상으로,
놓친 조건이 있는지 본문을
재검증하여 가산점 부여.



검색 엔진이 미처 잡지 못한
복합 혜택을 찾아내어, 사용자의
다중 의도를 완벽히 충족하는
카드를 최상위로 올림.

RAG 예시

나는 카페, 쇼핑, 주유, 간편결제 등의 혜택을 가진 카드를 추천받고 싶어

Best 추천 카드

BEST 추천



카드의정석
SHOPPING+

매칭 점수: 833점



💡 추천 이유 분석

🔥 Cafe(87.0)

Shopping(15.9)

🔥 Gas(96.1)

Payment(9.3)

주요 혜택

- [생활밀착 할인의 정석] 4대 주유소 주말 주유시 리터...
- [생활밀착 할인의 정석] 스타벅스, 풀바셋 10% 청구...
- [쇼핑 할인의 정석] 오프라인 쇼핑 10% 청구할인
- [쇼핑 할인의 정석] 온라인 쇼핑 10% 청구할인
- [추가 할인의 정석] 온라인 업종 4대 PAY 결제 시 5...

연회비: 10000

전월실적: 300000

두번째 추천 카드



카드의정석 I&U+

매칭 점수: 245점



💡 추천 이유 분석

🔥 Cafe(89.9)

🔥 Gas(98.4)

주요 혜택

- [Daily 할인] 주유 리터당 60~100원 청구할인
- [Daily 할인] 커피 10% 청구할인

연회비: 12000

전월실적: 300000

세번째 추천 카드



삼성 iD STATION 카드
(SK에너지)

매칭 점수: 225점



💡 추천 이유 분석

Shopping(7.1)

🔥 Gas(93.5)

Payment(11.9)

주요 혜택

- SK주유(충전)소에서 주유 시 10% 결제일할인
- 온라인쇼핑몰 5% 결제일할인
- 통신요금 정기결제 시 5% 결제일할인
- 편의점 5% 결제일할인

연회비: 15000

전월실적: 400000

RAG성능 평가 - 테스트 데이터셋 생성 및 구축

초기 평가셋:

“사용자가 입력할 법한 쿼리” 중심이었으나 정답셋을 3개로 제한하면서 실제 정답이 반영되지 않는 한계 발생

문제점: 예: “주유소 + 대형마트” 실제로 해당 혜택 카드가 더 많음에도 정답이 3개로 고정되어 정확한 추천이 MISS로 계산
될 수 있음



개선한 평가셋:

개선점:

카드 100개 각각의 혜택에서 4~5개 키워드 추출 → 키워드 기반 Complex 쿼리 생성 → 정답 = 해당 카드로 설정 → 정답-쿼리 간 정합성 강화

RAG성능 평가 - 평가 결과

```
35 {
36   "type": "Complex",
37   "query": "생활, 디지털구독, 아파트, 관리비, 통신 할인/적립을 같이 누릴 수 있는 카드",
38   "ground_truth_ids": [
39     "2885"
40   ],
41   "ground_truth": [
42     "2885"
43   ],
44   "target_name": "삼성 id SELECT ALL 카드"
45 },
46 {
47   "type": "Complex",
48   "query": "간편결제, 푸드, 통신, 디지털구독, 음식점 할인/적립을 같이 누릴 수 있는 카드",
49   "ground_truth_ids": [
50     "2441"
51   ],
52   "ground_truth": [
53     "2441"
54   ],
55   "target_name": "KB국민 My WE:SH 카드"
56 },
57 {
58   "type": "Complex",
59   "query": "간편결제, 디지털구독, 영상, 스트리밍, 디지털 혜택 다 있는 카드 추천해줘",
60   "ground_truth_ids": [
61     "2862"
62   ],
63   "ground_truth": [
64     "2862"
65   ],
66   "target_name": "MG+ S 하나카드"
67 },
68 {
69   "type": "Complex",
70   "query": "바우처, 공항라운지, 프리미엄 서비스, 트래블, 패션 할인/적립을 같이 누릴 수",
71   "ground_truth_ids": [
72     "2663"
73   ],
74   "ground_truth": [
75     "2663"
76   ],
```

◆ Complex Hit Rate@3 : 75.00% (75/100)

=====



"총 100개의 테스트 쿼리에 대해 평가를 진행한 결과,
상위 3개 추천 내 정답 포함 비율인 Hit Rate는 75.0%를 기록했으며,
평균적으로 정답 카드가 1.7번째 순위에 노출되는 0.60의 MRR 점수를
달성했습니다."

root@e5be7b3f57ec:/app/rag#

서비스 구성 및 구현 -메인 기능 소개

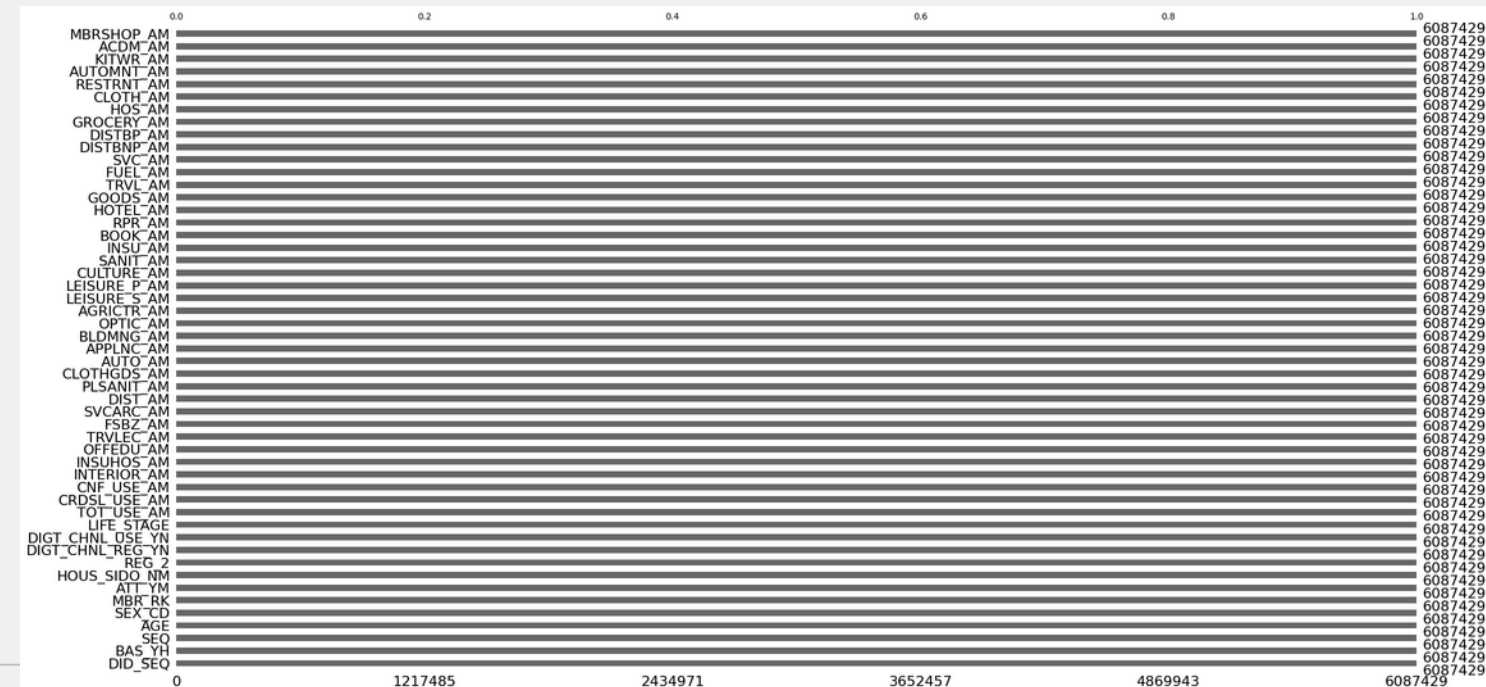
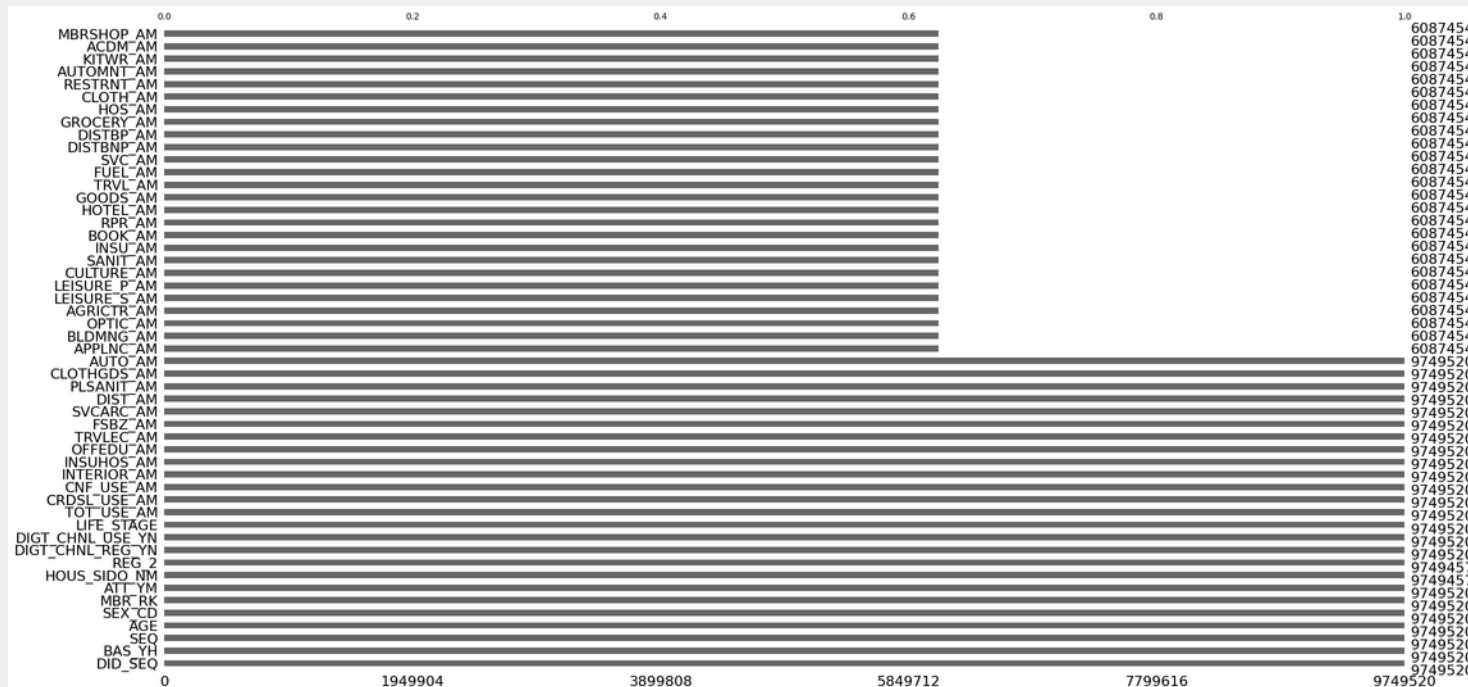
(4) ML - 데이터 전처리 및 피처 엔지니어링

0과 NaN만 포함하여 제거할 컬럼 'FUNITR_AM', 'HLTHFS_AM'.. 등 6개 & 결측치가 있는 행 → 제거 (6087429, 51)

```
columns_to_drop = []

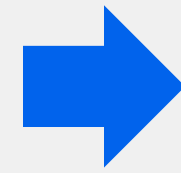
for col in df.columns:
    # 결측치(NaN)를 제외한 고유값 확인
    unique_values_excluding_nan = df[col].dropna().unique()

    # 고유값이 없거나 (모든 값이 NaN), 유일하게 0만 있다면 제거 대상
    if len(unique_values_excluding_nan) == 0 or \
       (len(unique_values_excluding_nan) == 1 and unique_values_excluding_nan[0] == 0.0):
        columns_to_drop.append(col)
```



(4) ML - 데이터 전처리 및 피처 엔지니어링

SEQ(고객번호) 2년 동안의 총 8 분기의 데이터가
모두 존재하는 고객의 데이터만 사용



총 고객(SEQ) 수: 524703명
총 행(레코드) 수: 4197624개 (524703 * 8)

```
{seq_counts[seq_counts['행_개수'] == 8].shape[0]}
```

```
# 고객(SEQ) 별로 존재하는 고유한 분기(BAS_YH)의 개수를 계산합니다.  
# 결과는 각 SEQ별 분기 개수를 담고 있는 시리즈가 됩니다.  
quarter_counts = df.groupby('SEQ')['BAS_YH'].nunique()
```

```
full_data_seqs = quarter_counts[quarter_counts == 8].index
```

```
df_full_data = df[df['SEQ'].isin(full_data_seqs)]
```

```
## 8개의 행이 존재하는 고객 524,703명  
8*524703
```

```
4197624
```

(4) ML - 데이터 전처리 및 피처 엔지니어링

신용카드 이용금액(CRDSL_USE_AM)이 0보다 크고, 8분기 중 모든 분기에 신용카드 이용금액이 있는 고객의 데이터만을 필터링

```
# 데이터 필터링 (Row Level)
# 신용카드 이용금액(CRDSL_USE_AM)이 0보다 큰 내역만 남김
# (체크카드만 쓴 분기 데이터는 신용카드 분석에 노이즈가 되므로 제거)
df_credit = df_full[df_full['CRDSL_USE_AM'] > 0].copy()

print(f"  - 전체 행: {len(df_full)}")
print(f"  - 신용카드 사용 행: {len(df_credit)} (삭제된 행: {len(df_full) - len(df_credit)})")

- 전체 행: 4197624
- 신용카드 사용 행: 3498992 (삭제된 행: 698632)

# 8분기 모두 실적이 있는 '충성 고객'만 필터링
seq_counts = df_credit.groupby('SEQ').size()
loyal_seqs = seq_counts[seq_counts == 8].index
df_credit_final = df_credit[df_credit['SEQ'].isin(loyal_seqs)].copy()

print(f"  - (신용카드 사용): {len(df_credit)}행")
print(f"  - (8분기 완주자): {len(df_credit_final)}행 (고객 수: {len(loyal_seqs)}명)")

- (신용카드 사용): 3498992행
- (8분기 완주자): 3205312행 (고객 수: 400664명)

df_credit_final.shape

(3205312, 51)
```



(4) ML - 데이터 전처리 및 피처 엔지니어링

금액 관련(_AM) 칼럼 중 총이용금액, 체크카드이용금액, 중분류의 합으로 계산 가능한 대분류(10개) 제외.

정적칼럼에서는 나이, 성별, 라이프스테이지 유지.

```
# 제거할 대분류 이용 금액 컬럼 리스트
large_category_am_cols = [
    'INTERIOR_AM', 'INSUHOS_AM', 'OFFEDU_AM', 'TRVLEC_AM', 'FSBZ_AM',
    'SVCARC_AM', 'DIST_AM', 'PLSANIT_AM', 'CLOTHGDS_AM', 'AUTO_AM'
]
```

```
# df에서 해당 컬럼들을 제거하여 df_selected 생성
df_selected = df.drop(columns=large_category_am_cols)
```

```
# 컬럼 정의
# 행동 특징 (신용카드 기준)
am_cols = [
    'CRDSL_USE_AM',
    'APPLNC_AM', 'BLDMNG_AM', 'OPTIC_AM', 'AGRICTR_AM', 'LEISURE_S_AM',
    'LEISURE_P_AM', 'CULTURE_AM', 'SANIT_AM', 'INSU_AM', 'BOOK_AM',
    'RPR_AM', 'HOTEL_AM', 'GOODS_AM', 'TRVL_AM', 'FUEL_AM', 'SVC_AM',
    'DISTBNP_AM', 'DISTBP_AM', 'GROCERY_AM', 'HOS_AM', 'CLOTH_AM',
    'RESTRNT_AM', 'AUTOMNT_AM', 'KITWR_AM', 'ACDM_AM', 'MBRSHOP_AM'
]

# 정적 특징
static_cols = ['AGE', 'SEX_CD', 'LIFE_STAGE']
```

(4) ML 기능 - 데이터 전처리 및 피처 엔지니어링

금액 관련(_AM) 칼럼에 ['mean', 'std', 'median'] 를 각각 적용하여
8개 분기의 데이터를 하나의 행으로 압축



29개의 AM칼럼은 $29 \times 3 = 87$ 개로 증가

```
# 특징 생성 (Aggregation)
df_behavioral = df_credit_final.groupby('SEQ')[am_cols].agg(['mean', 'std', 'median'])
```

RESTRNT_AM : 일반/휴게음식 소비금액



RESTRNT_AM_Mean
RESTRNT_AM_StdDev
RESTRNT_AM_Median

(4) ML 기능 - 데이터 전처리 및 피처 엔지니어링

"TOP_CAT_MEAN' 변수 생성' - 가장 많이 쓴 분야
금액관련 칼럼 중 '신용카드 전체 사용 금액', '유통 소비 금액'
을 제외한 가장 많이 쓴 분야의 소비 금액

'VOLATILITY_RATIO' 변수 생성 - 신용카드 지출 변동성

8분기 신용카드 사용 금액의 표준편차 / 8분기 신용카드 사용 금액 평균

```
# 파생 변수 (변동성, 선호업종)
df_behavioral['VOLATILITY_RATIO'] = df_behavioral['CRDSL_USE_AM_StdDev'] / df_behavioral['CRDSL_USE_AM_Mean']
target_means = [c for c in df_behavioral.columns if c.endswith('_Mean') and c != 'CRDSL_USE_AM_Mean' and 'DISTBP' not in c]
df_behavioral['TOP_CAT_MEAN'] = df_behavioral[target_means].idxmax(axis=1).str.replace('_Mean', '')
```

(4) ML 기능 - 데이터 전처리 및 피처 엔지니어링

[나이, 성별, 라이프스테이지, 최다소비분야]
범주형 칼럼



OneHotEncoder

금액 관련(_AM) 칼럼



로그변환 + StandardScaler

```
# 파이프라인
preprocessor = ColumnTransformer(
    transformers=[
        ('log_num', Pipeline(steps=[('log', FunctionTransformer(np.log1p)), ('scaler', StandardScaler())])), log_cols
        ('num', Pipeline(steps=[('scaler', StandardScaler())])), other_num_cols,
        ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False), cat_cols)
    ]
)

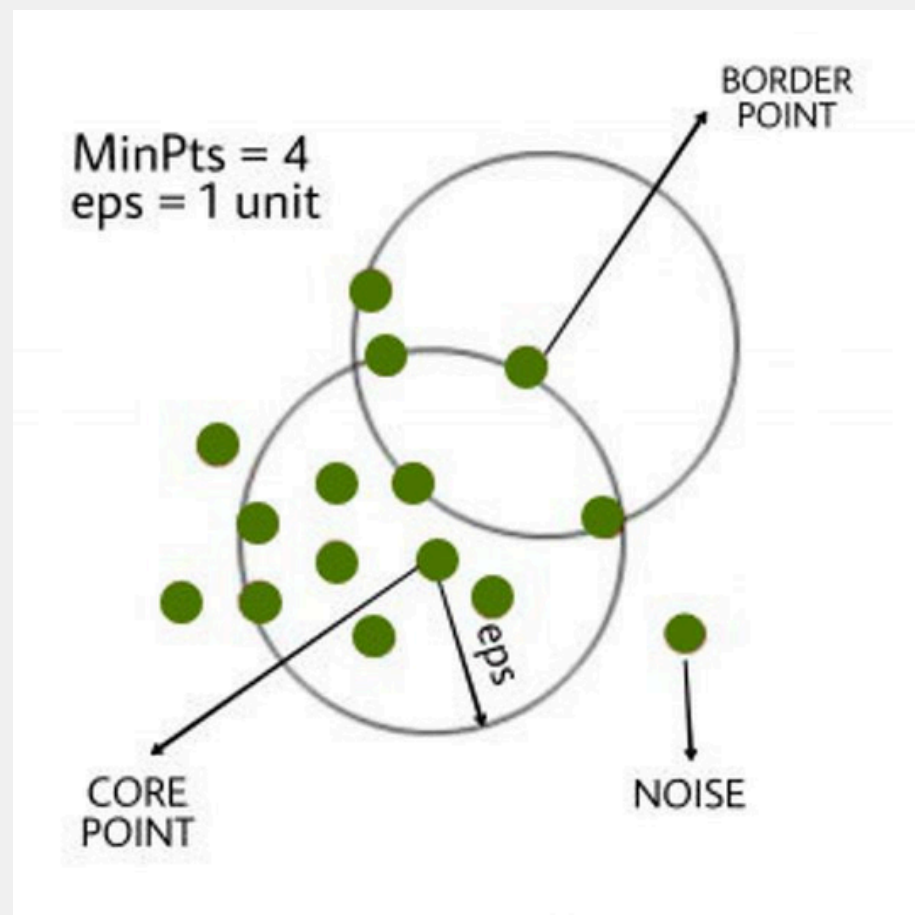
X_processed = preprocessor.fit_transform(df_clustering)
```

(4) ML 기능 - 클러스터링

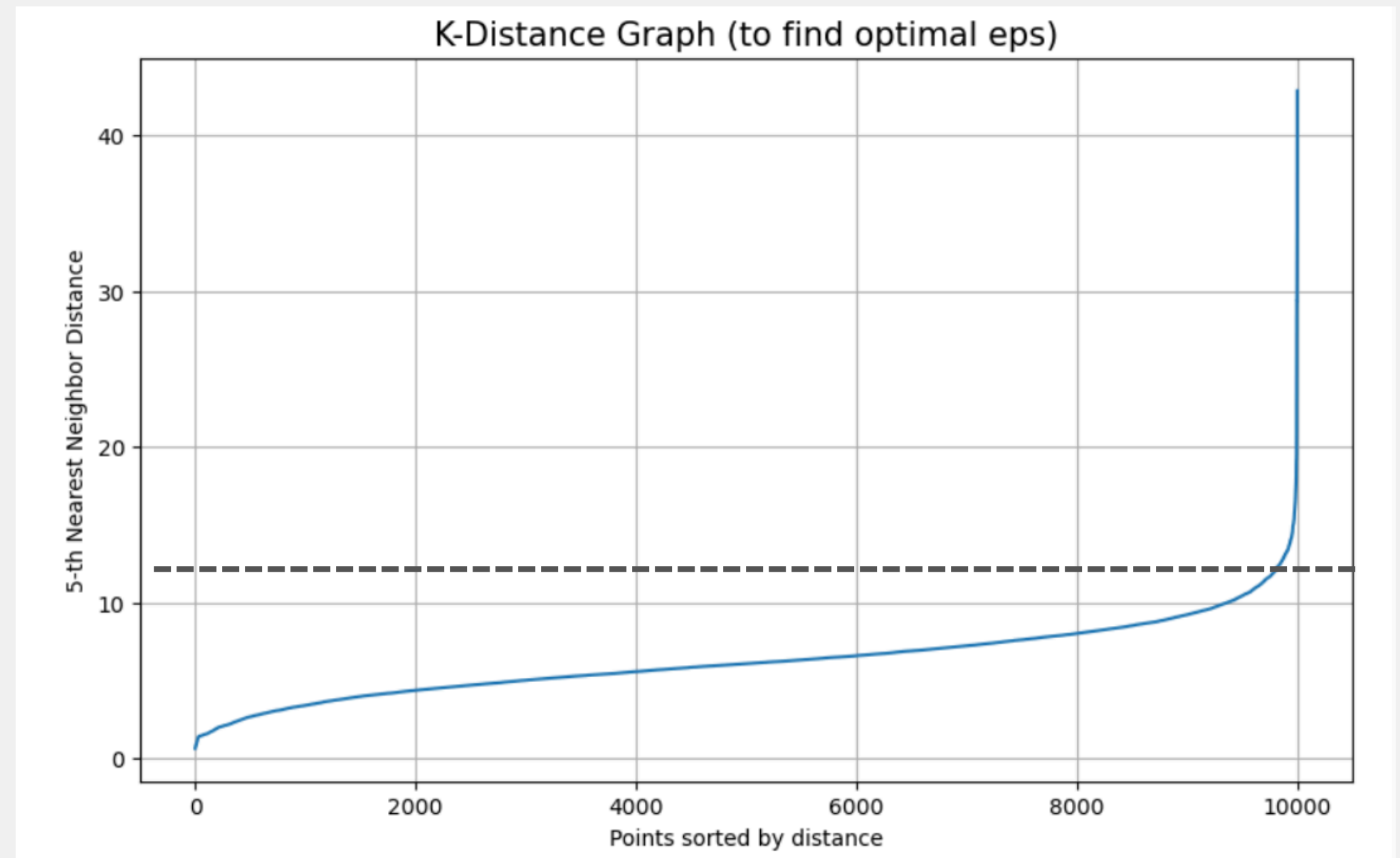
밀도 기반 클러스터링 DBSCAN

특정 데이터 포인트를 중심으로 몇 개의 데이터 포인트가 있는지 찾기 위한 반경 (반지름)

EPS



K-Distance Graph를 통한 EPS값 범위 설정



11 ~ 15 부근으로 EPS를 설정

(4) ML 기능 - 클러스터링

밀도 기반 클러스터링 DBSCAN

탐색할 파라미터 후보군 설정

```
eps_values = [10.5, 11.0, 11.5, 12.0, 12.5, 13.0]
```

```
min_samples_values = [5, 10, 15, 20]
```

3. 조합별 실행

```
for eps, min_samples in product(eps_values, min_samples_values):
```

```
    # DBSCAN 실행
```

```
    dbscan = DBSCAN(eps=eps, min_samples=min_samples, n_jobs=-1)
```

```
    labels = dbscan.fit_predict(X_sample)
```

```
    # 지표 계산
```

```
    n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
```

```
    n_noise = list(labels).count(-1)
```

```
    noise_ratio = n_noise / len(labels) * 100
```

```
    # 클러스터가 2개 이상일 때만 실루엣 계수 계산 (1개면 계산 불가)
```

```
    if n_clusters > 1:
```

```
        score = silhouette_score(X_sample, labels)
```

```
    else:
```

```
        score = -1 # 실패 처리
```

Grid Search 진행 중 (eps x min_samples)...

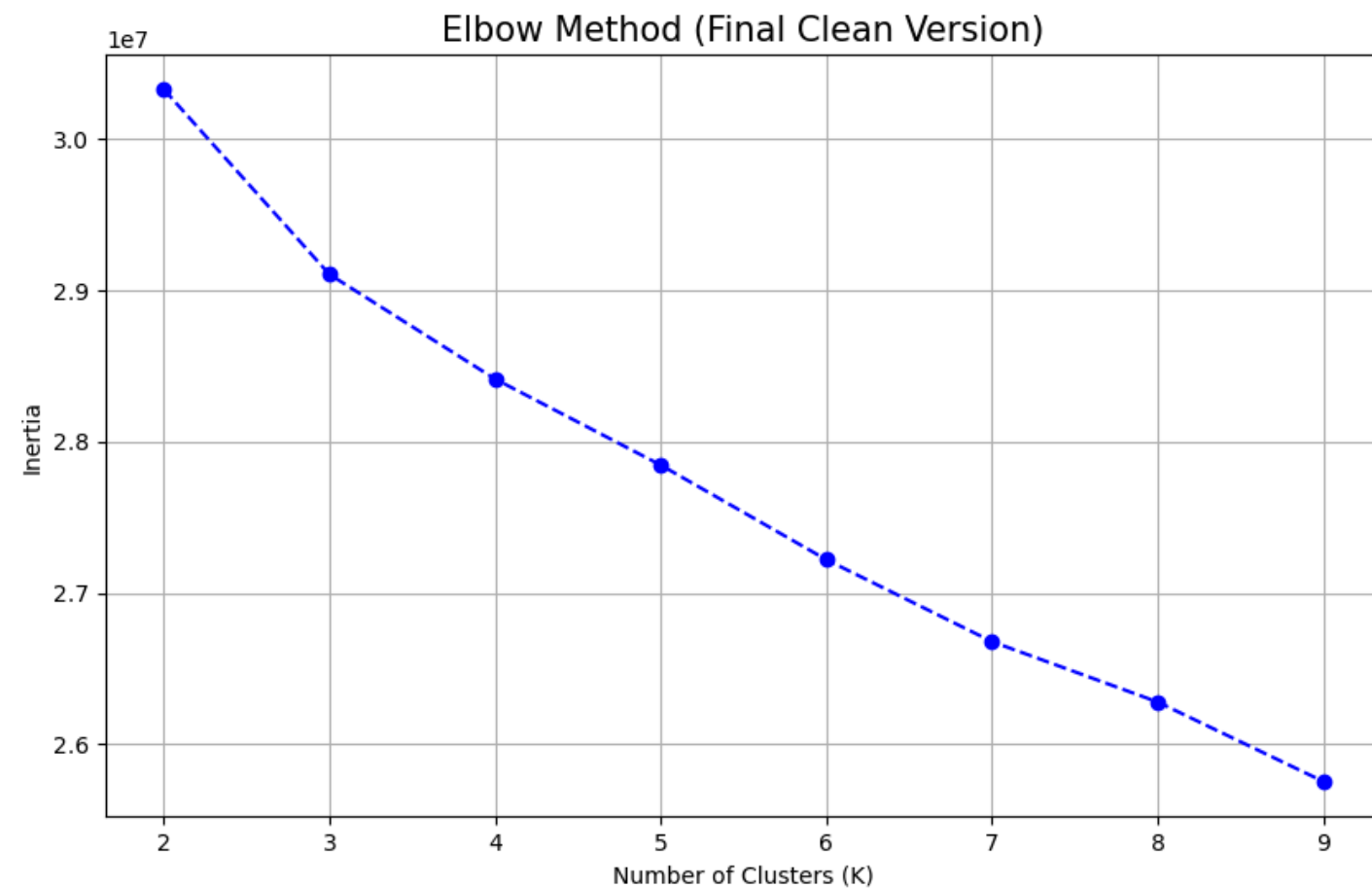
eps	min_s	Clusters	Noise(%)	Silhouette
10.5	5	2	3.0	0.409
10.5	10	1	3.4	-1.000
10.5	15	1	3.5	-1.000
10.5	20	1	3.7	-1.000
11.0	5	2	2.3	0.429
11.0	10	1	2.5	-1.000
11.0	15	1	2.6	-1.000
11.0	20	1	2.7	-1.000
11.5	5	2	1.6	0.458
11.5	10	1	1.9	-1.000
11.5	15	1	2.0	-1.000
11.5	20	1	2.1	-1.000
12.0	5	2	1.2	0.479
12.0	10	1	1.5	-1.000
12.0	15	1	1.5	-1.000
12.0	20	1	1.5	-1.000
12.5	5	2	0.8	0.494
12.5	10	2	1.0	0.498
12.5	15	1	1.2	-1.000
12.5	20	1	1.2	-1.000
13.0	5	2	0.6	0.524
13.0	10	2	0.7	0.519
13.0	15	1	0.9	-1.000
13.0	20	1	1.0	-1.000

클러스터의 개수가 1개로
설정되어 사용할 수 없는 군집화

클러스터의 개수가 2개일 경우
실루엣 계수는 높지만 비즈니스
적 활용 가치가 낮음.

(4) ML 기능 - 클러스터링

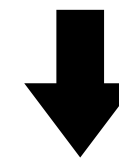
K-Means에 적합한 k를 찾기위한 Elbow Method를 수행



x축: 군집의 개수

y축: Inertia (군집 내 오차의 합)

급격하기 꺾이는 지점 이후 완만해지는 지점

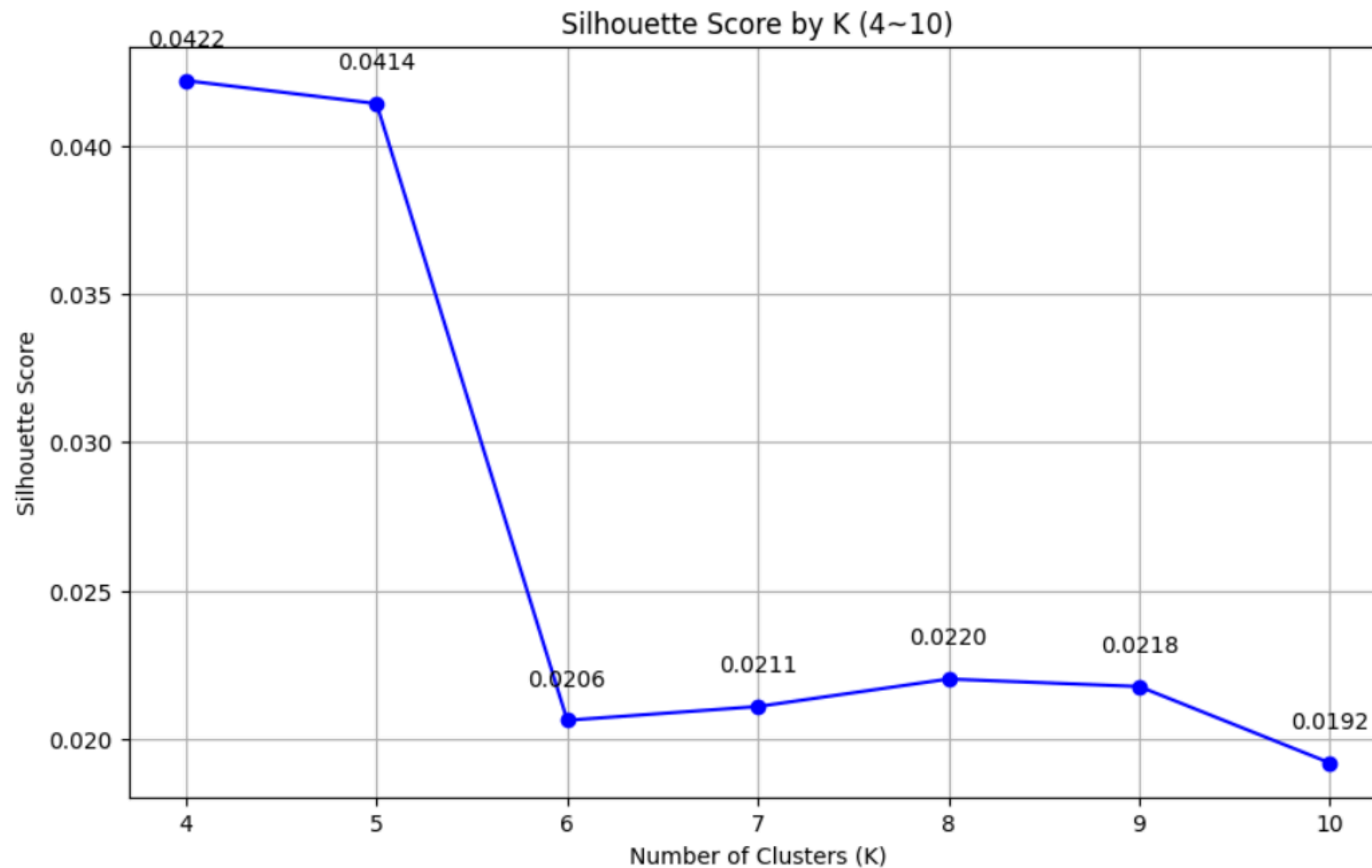


군집의 수를 늘려도 성능 개선 효과가 미비

그러나 결과는 elbow지점이 정확하지 않음

(4) ML 기능 - 클러스터링

군집의 개수를 4에서 10개로 하여 실루엣 계수를 계산



실루엣 계수란? $s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$

$a(i)$: 같은 클러스터 내 다른 점들과의 평균 거리

$b(i)$: 가장 가까운 다른 클러스터와의 평균 거리

군집 내 응집도와 군집 간 차이를 동시에 고려

k = 6 이후부터 급격히 하락하는 계수

(4) ML 기능 - 클러스터링

군집의 개수를 4, 5개로 설정하여 k-means수행

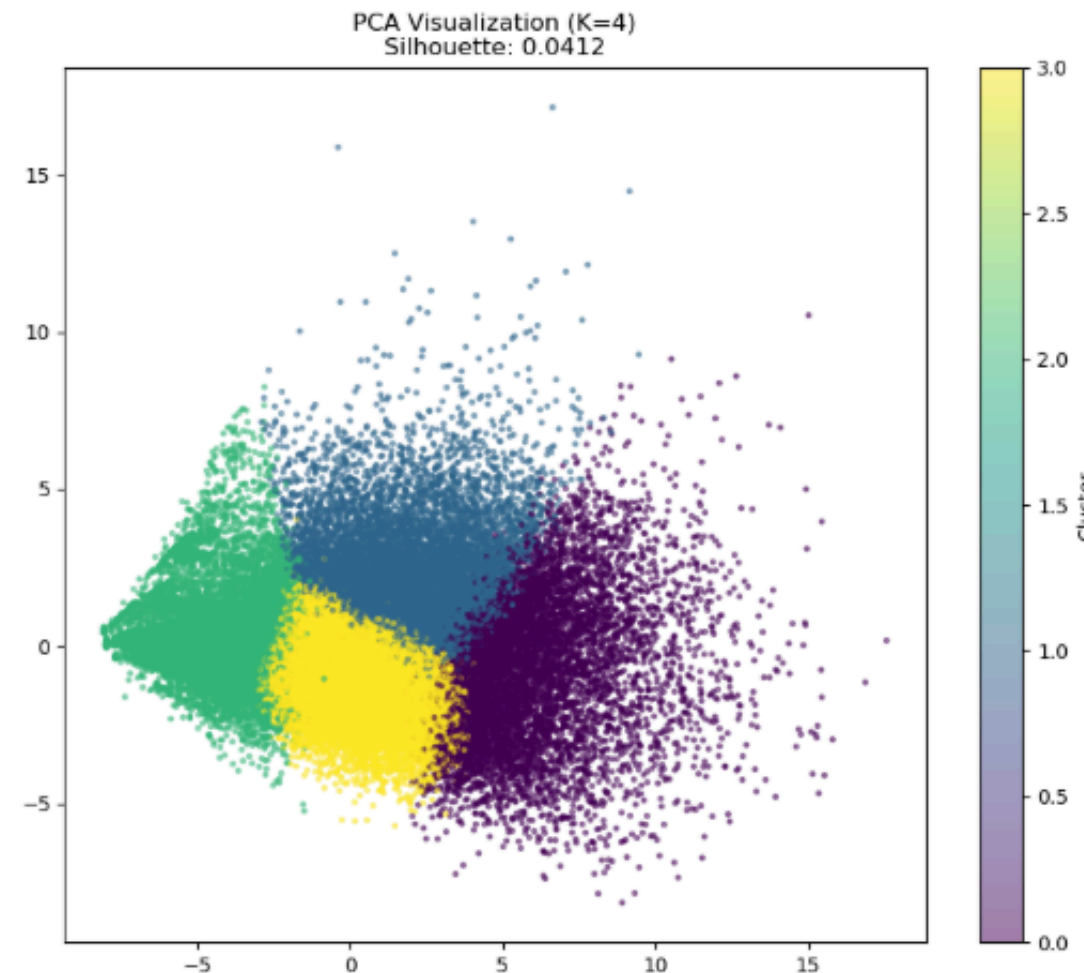
```
# K-Means 클러스터링 (K=4, 5)
kmeans_k4 = KMeans(n_clusters=4, random_state=42, n_init=10).fit(X_processed)
df_features['Cluster_K4'] = kmeans_k4.labels_

kmeans_k5 = KMeans(n_clusters=5, random_state=42, n_init=10).fit(X_processed)
df_features['Cluster_K5'] = kmeans_k5.labels_
```

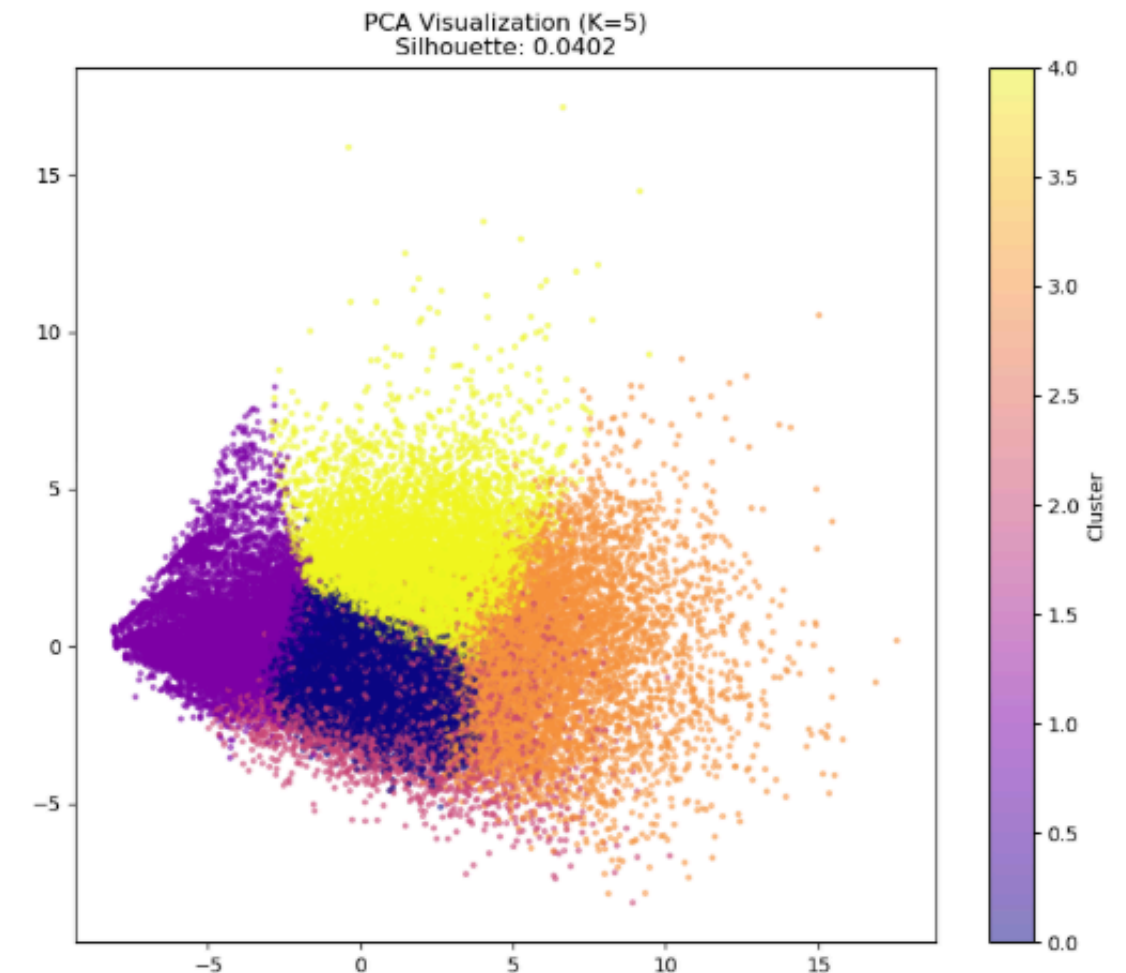
PCA를 통해 2차원에
클러스터링 된 데이터를 표현

주성분 분석(PCA)이란?
고차원 데이터를 저차원으로 변환하여
데이터의 분산을 최대한 보존하는 방식
으로 차원을 축소하는 기법

PCA 변환 및 시각화 생성 중...



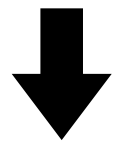
Silhouette score: 0.0425



Silhouette score: 0.0417

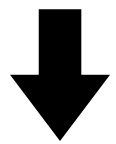
(4) ML 기능 - 클러스터링

PC1: 모든 변수가 양의 방향



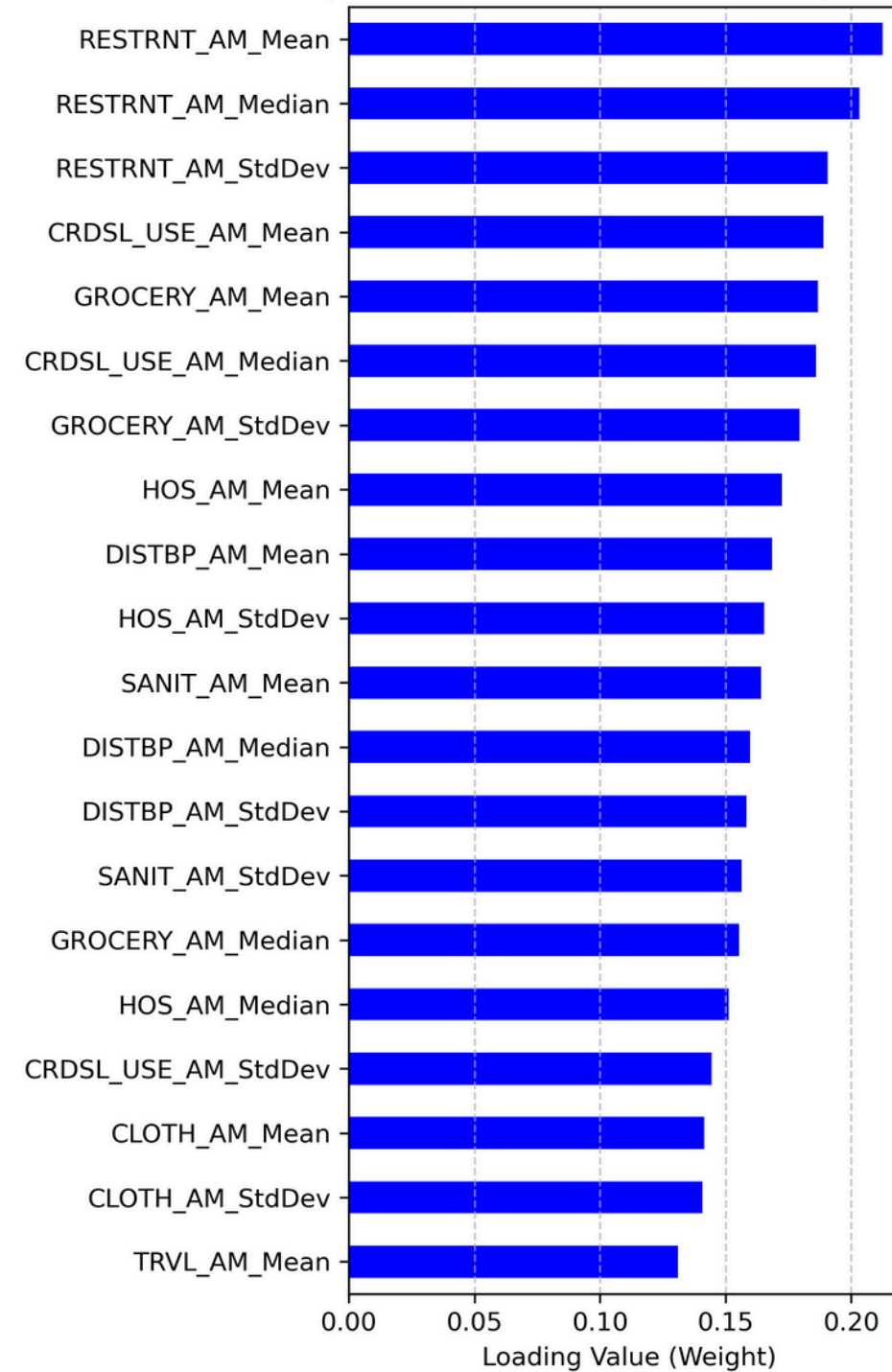
소비규모

PC2: 양의 방향과 음의 방향
이 동시에 존재

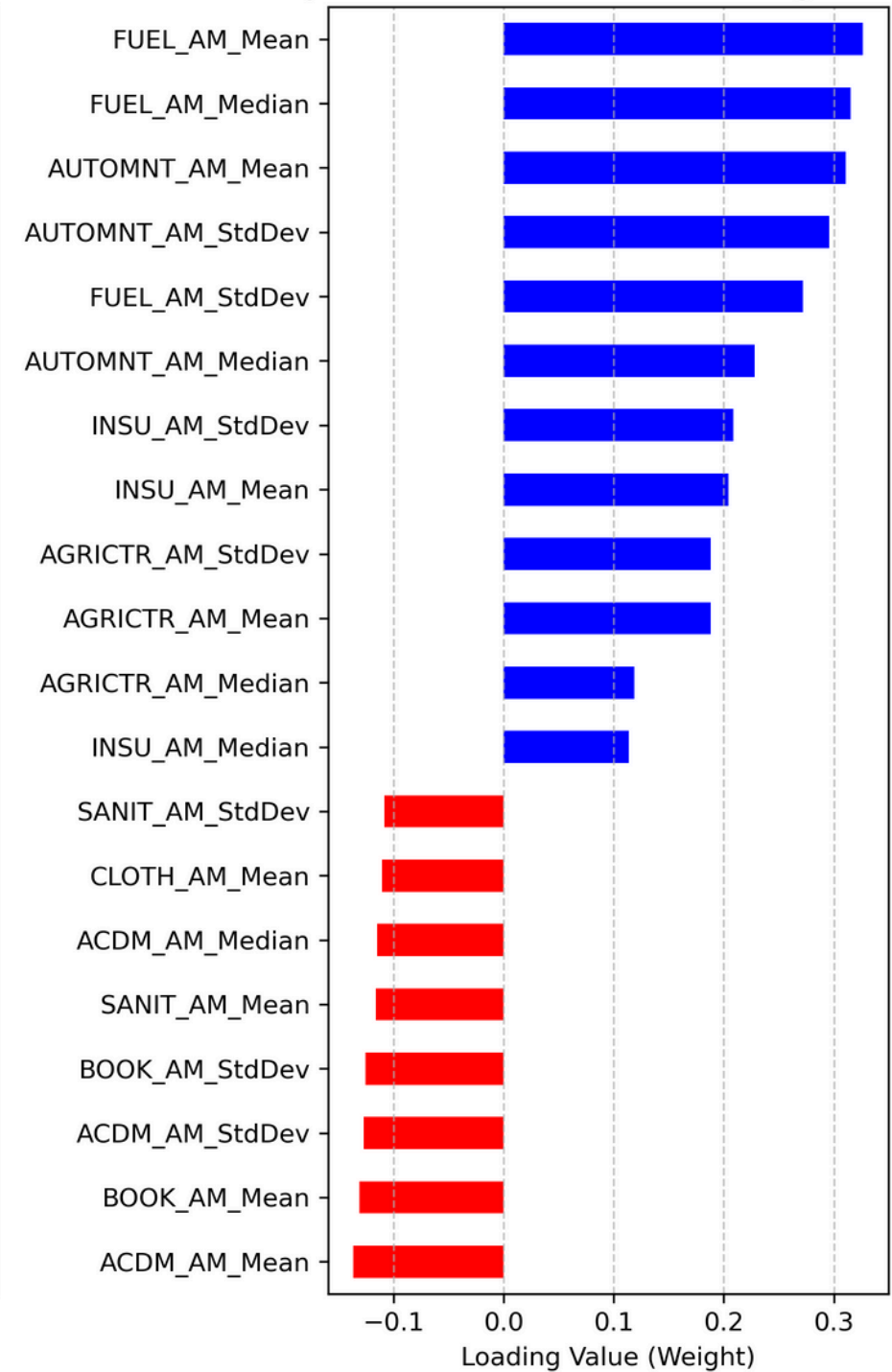


소비성향

Top 20 Features Contributing to PC1

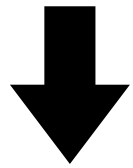
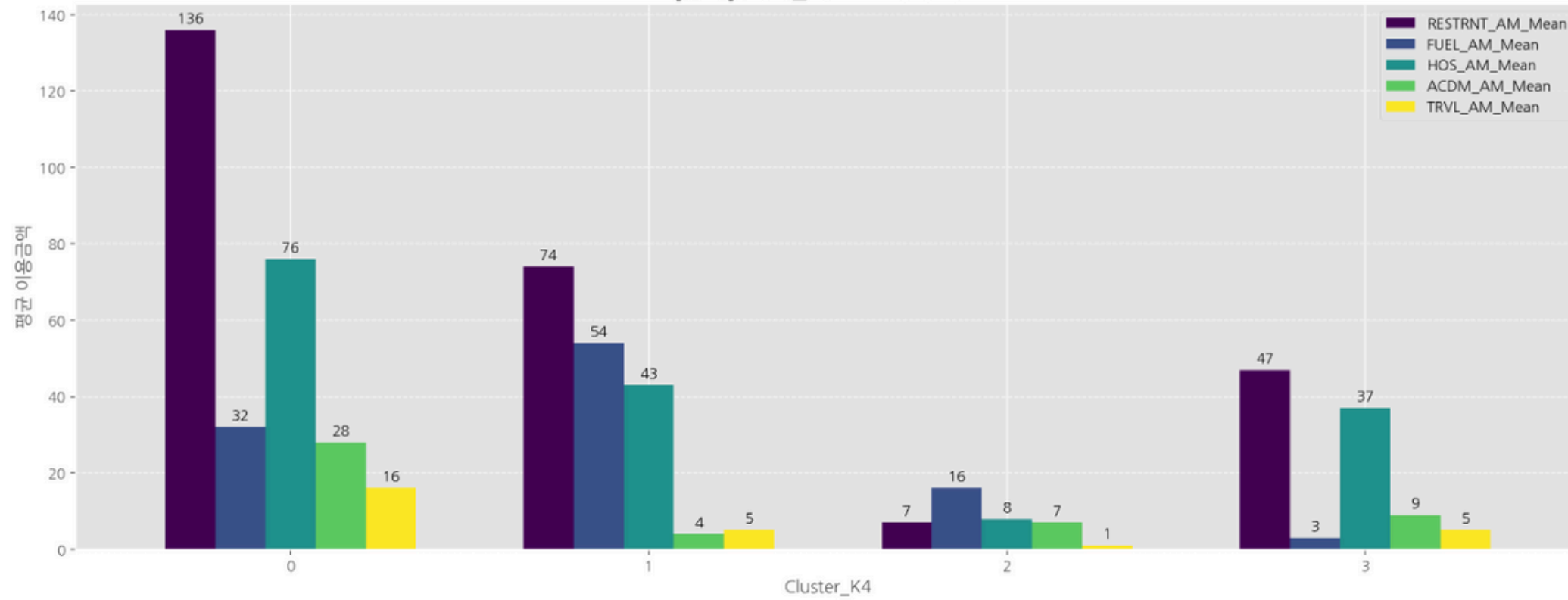


Top 20 Features Contributing to PC2

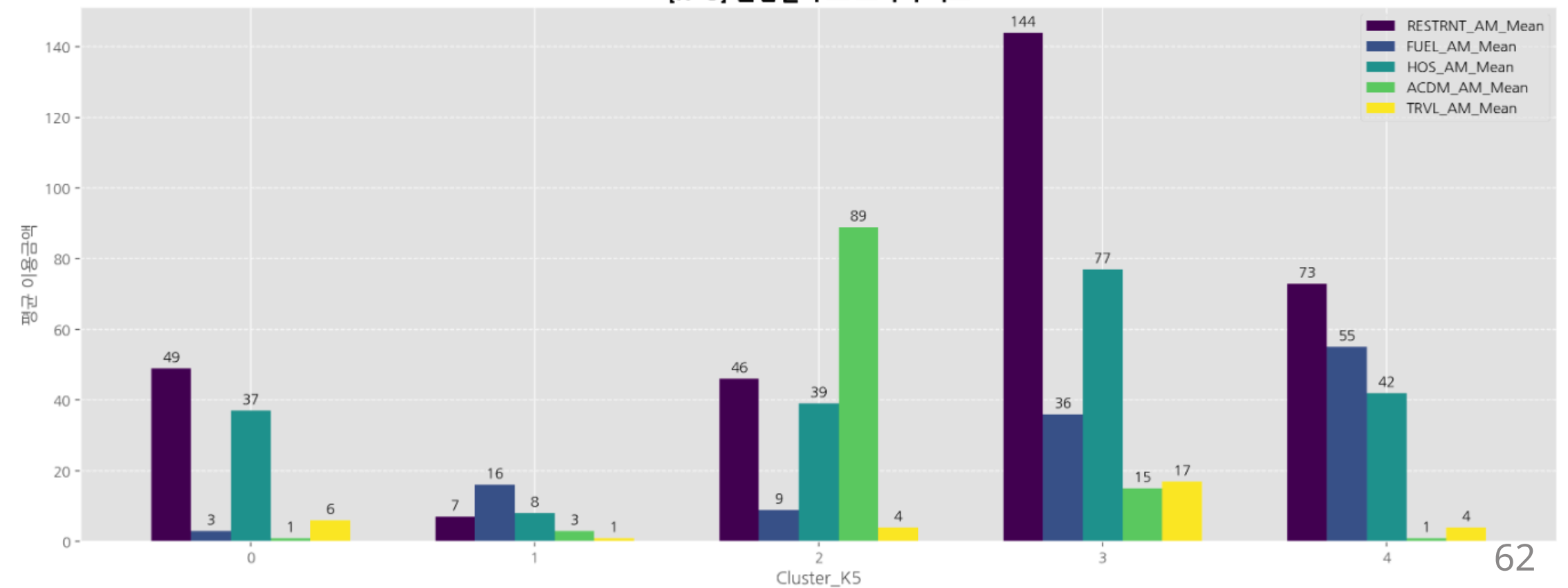


(4) ML 기능 - 클러스터링

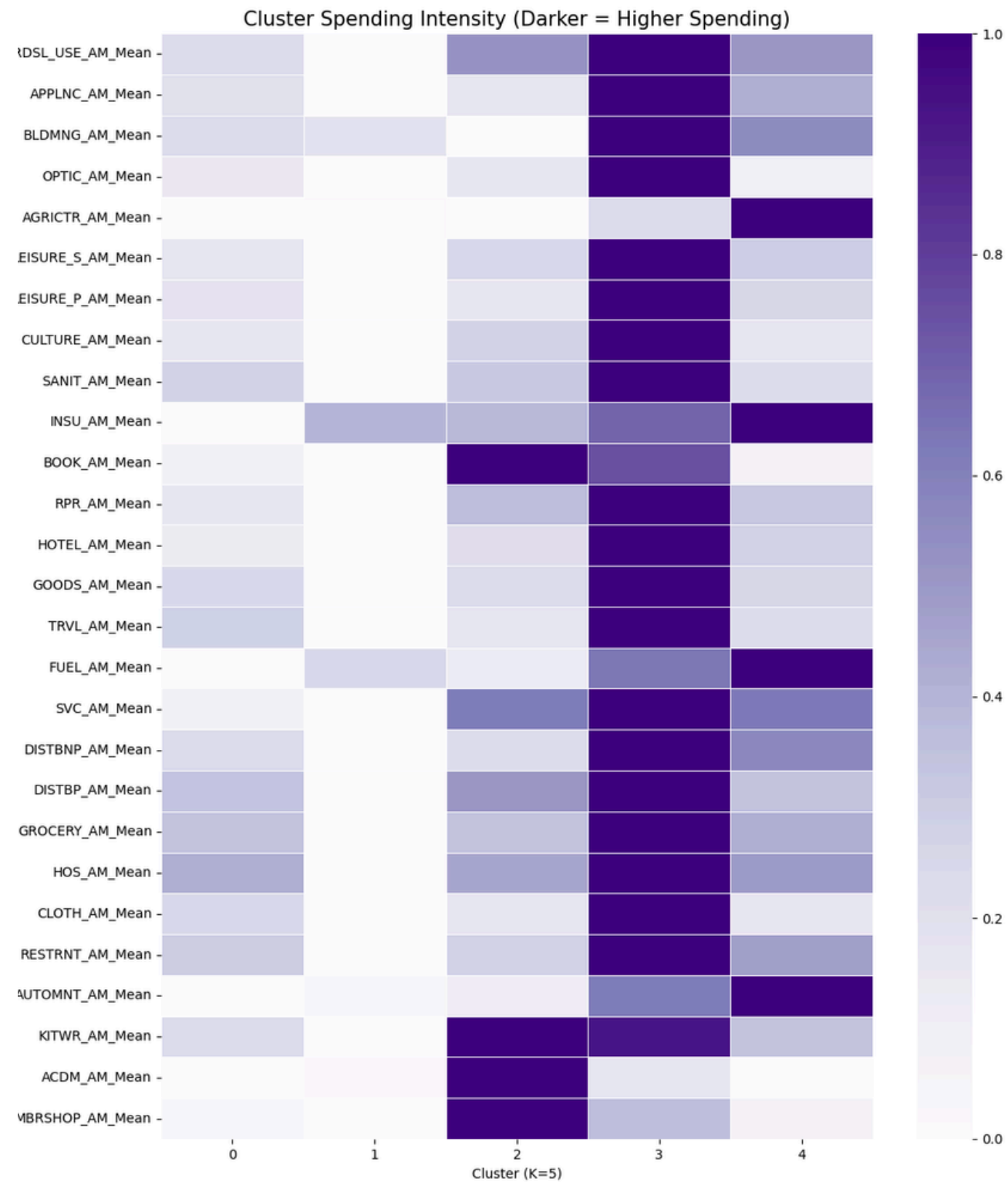
[K=4] 군집별 주요 소비처 비교



[K=5] 군집별 주요 소비처 비교



(4) ML 기능 - 클러스터링



K=5 클러스터 통합 프로파일링

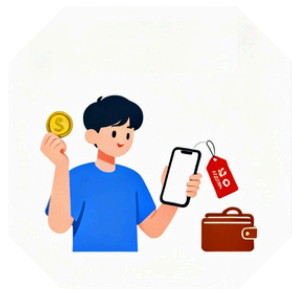
	Cluster 0 (33.7%)	Cluster 1 (27.5%)	Cluster 2 (7.7%)	Cluster 3 (13.2%)	Cluster 4 (17.9%)
LIFE_STAGE	SECLIFE	CHILD_UNI	CHILD_TEEN	CHILD_TEEN	CHILD_UNI
TOP_CAT_MEAN	RESTRNT_AM	SVC_AM	ACDM_AM	RESTRNT_AM	RESTRNT_AM
SEX_CD	2	2	2	2	2
CRDSL_USE_AM_Mean	363.8	201.4	572.8	915.1	564.1
APPLNC_AM_Mean	0.3	0.1	0.3	1.1	0.5
BLDMNG_AM_Mean	7.3	7.1	5.7	13.3	9.9
OPTIC_AM_Mean	0.2	0.0	0.2	1.2	0.2
AGRICTR_AM_Mean	0.0	0.0	0.0	0.2	0.8
LEISURE_S_AM_Mean	3.7	0.7	5.0	18.0	5.8
LEISURE_P_AM_Mean	1.0	0.2	0.9	4.5	1.3
CULTURE_AM_Mean	0.5	0.1	0.8	2.8	0.5
SANIT_AM_Mean	4.9	0.7	5.5	16.3	4.0
INSU_AM_Mean	7.1	13.7	13.5	18.6	23.7
BOOK_AM_Mean	0.3	0.1	2.2	1.7	0.2
RPR_AM_Mean	0.3	0.1	0.6	1.5	0.5
HOTEL_AM_Mean	0.5	0.1	0.8	3.3	1.0
GOODS_AM_Mean	0.7	0.1	0.7	2.4	0.7
TRVL_AM_Mean	5.7	1.1	3.9	16.9	4.5
FUEL_AM_Mean	3.2	16.1	9.5	36.0	54.6
SVC_AM_Mean	17.5	15.8	29.9	38.7	30.1
DISTBNP_AM_Mean	2.5	0.5	2.3	8.7	5.2
DISTBP_AM_Mean	183.5	77.2	237.9	390.7	185.3
GROCERY_AM_Mean	9.5	1.0	9.5	25.6	11.5
HOS_AM_Mean	37.0	7.9	39.2	76.7	41.9
CLOTH_AM_Mean	3.4	0.5	2.4	11.9	2.3
RESTRNT_AM_Mean	48.9	6.7	45.8	144.3	72.6
AUTOMNT_AM_Mean	0.6	1.2	2.0	8.6	13.5
KITWR_AM_Mean	0.7	0.5	1.1	1.1	0.7
ACDM_AM_Mean	1.0	3.4	89.3	15.4	1.5
MBRSHOP_AM_Mean	0.1	0.0	1.0	0.4	0.1
AGE	47.1	48.2	42.9	44.0	49.5
VOLATILITY_RATIO	0.5	0.5	0.4	0.4	0.4

(4) ML 기능 - 클러스터링

알뜰 소액족

소비금액: 중하위

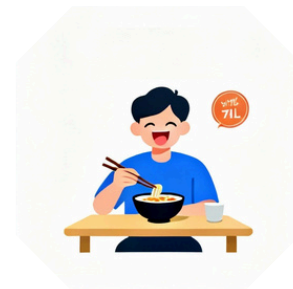
대중교통을 이용,
외식과 건강 관리에
집중하는 실속형 소
비자



실속 미식가

소비금액: 최하위

소비를 극도로 자제
하며, 필수적인 차량
유지비나 공과금/보
험료 납부 위주로 카
드를 사용하는 '짠테
크' 성향



에듀 맘/대디

소비금액: 중상위

교육에 올인하는
고객.
외식과 병원지출도
준수하여
전반적으로 활발한
소비 활동을 함.



럭셔리 VIP

소비금액: 최상위

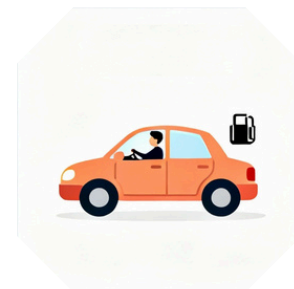
경제적 여유가
넘치며, 외식, 여행,
레저 등 '삶의 질'을
높이는 데 돈을 아끼
지 않는 우수 고객
(VIP)



마이카 중산층

소비금액: 중상위

자차를 보유하고 적
극적으로 관리/정비
하는 고객입니다.
외식, 보험 지출도
높아 안정적인 경제
력을 갖춘
중산층 고객.



(4) ML 기능 - 분류모델

Decision Tree의 Threshold

For Cold Start User

질문별 기준 금액 설정을 위한 분포 분석

사분위수

Q2_CAR (주유)

- 전체 고객 중 이용자 비율: 53.4%
- [실사용자 기준] 중앙값(Med): 15 | 하위25%: 4 | 상위25%: 44
- 추천 기준점: 15 이상이면 'Yes/High'

Q3_DINING (외식)

- 전체 고객 중 이용자 비율: 86.5%
- [실사용자 기준] 중앙값(Med): 39 | 하위25%: 14 | 상위25%: 86
- 추천 기준점: 39 이상이면 'Yes/High'

Q4_LEISURE (여행+레저)

- 전체 고객 중 이용자 비율: 56.7%
- [실사용자 기준] 중앙값(Med): 10 | 하위25%: 4 | 상위25%: 24
- 추천 기준점: 10 이상이면 'Yes/High'

Q5_EDU (학원)

- 전체 고객 중 이용자 비율: 18.8%
- [실사용자 기준] 중앙값(Med): 25 | 하위25%: 8 | 상위25%: 80
- 추천 기준점: 25 이상이면 'Yes/High'

Q_HEALTH (병원)

- 전체 고객 중 이용자 비율: 79.1%
- [실사용자 기준] 중앙값(Med): 25 | 하위25%: 9 | 상위25%: 60
- 추천 기준점: 25 이상이면 'Yes/High'

Q1_SPEND (CRDSL_USE_AM_Mean)

추천 기준선: [131, 219, 638]

```

|--- Value <= 219.38
|   |--- Value <= 131.88
|   |   |--- class: 1
|   |--- Value > 131.88
|   |   |--- class: 1
|--- Value > 219.38
|   |--- Value <= 638.12
|   |   |--- class: 0
|   |--- Value > 638.12
|   |   |--- class: 3

```

Q2_CAR (FUEL_AM_Mean)

추천 기준선: [0, 9, 23]

```

|--- Value <= 9.38
|   |--- Value <= 0.62
|   |   |--- class: 0
|   |--- Value > 0.62
|   |   |--- class: 0
|--- Value > 9.38
|   |--- Value <= 23.12
|   |   |--- class: 4
|   |--- Value > 23.12
|   |   |--- class: 4

```

Q3_DINING (RESTRNT_AM_Mean)

추천 기준선: [0, 6, 86]

```

|--- Value <= 6.88
|   |--- Value <= 0.62
|   |   |--- class: 1
|   |--- Value > 0.62
|   |   |--- class: 1
|--- Value > 6.88
|   |--- Value <= 86.88
|   |   |--- class: 0
|   |--- Value > 86.88
|   |   |--- class: 3

```

Q4_LEISURE (TRVL_AM_Mean + LEISURE_S_AM_Mean)

추천 기준선: [0, 24]

```

|--- Value <= 0.62
|   |--- class: 1
|--- Value > 0.62
|   |--- Value <= 24.38
|   |   |--- class: 0
|   |--- Value > 24.38
|   |   |--- class: 3

```

Q5_EDU (ACDM_AM_Mean)

추천 기준선: [0, 18, 34]

```

|--- Value <= 18.12
|   |--- Value <= 0.62
|   |   |--- class: 0
|   |--- Value > 0.62
|   |   |--- class: 0
|--- Value > 18.12
|   |--- Value <= 34.38
|   |   |--- class: 2
|   |--- Value > 34.38
|   |   |--- class: 2

```

(4) ML 기능 - 분류모델

For Cold Start User

외식을 얼마나 자주 가시나요?

RESTRNT_AM_Mean:100

→

Q_DINING = 3_High

```
# Q1. 소비 규모 (기준: 22만 / 64만)
conditions_spend = [
    (df_sim['CRDSL_USE_AM_Mean'] < 220),
    (df_sim['CRDSL_USE_AM_Mean'] >= 220) & (df_sim['CRDSL_USE_AM_Mean'] < 640),
    (df_sim['CRDSL_USE_AM_Mean'] >= 640)
]
df_sim['Q_SPEND'] = np.select(conditions_spend, ['1_Low', '2_Mid', '3_High'], default='2_Mid')

# Q2. 자차 (기준: 1만)
df_sim['Q_CAR'] = np.where(df_sim['FUEL_AM_Mean'] >= 10, 'Yes', 'No')

# Q3. 외식 (기준: 1만 / 9만)
conditions_dining = [
    (df_sim['RESTRNT_AM_Mean'] < 10),
    (df_sim['RESTRNT_AM_Mean'] >= 10) & (df_sim['RESTRNT_AM_Mean'] < 90),
    (df_sim['RESTRNT_AM_Mean'] >= 90)
]
df_sim['Q_DINING'] = np.select(conditions_dining, ['1_Low', '2_Mid', '3_High'], default='2_Mid')

# Q4. 여행/레저 (기준: 2.5만)
total_leisure = df_sim['TRVL_AM_Mean'] + df_sim['LEISURE_S_AM_Mean']
df_sim['Q_LEISURE'] = np.where(total_leisure >= 25, 'Yes', 'No')

# Q5. 교육 (기준: 3.5만)
df_sim['Q_EDU'] = np.where(df_sim['ACDM_AM_Mean'] >= 35, 'Yes', 'No')

# Q6. 병원 (기준: 2만)
df_sim['Q_HEALTH'] = np.where(df_sim['HOS_AM_Mean'] >= 20, 'Yes', 'No')
```

(4) ML 기능 - 분류모델



For Cold Start User

```
clf_setup = setup(  
    data=data_for_automl,  
    target='Cluster_K5',  
    session_id=42,  
  
    # 불균형 처리  
    fix_imbalance=True,  
    fix_imbalance_method=SMOTE(random_state=42),  
  
    # 전처리  
    normalize=True,  
    categorical_features=['AGE', 'SEX_CD', 'LIFE_STAGE', 'Q_SPEND', 'Q_CAR', 'Q_DINING', 'Q_LEISURE', 'Q_EDU', 'Q_HEALTH'],  
  
    # MLflow 자동 로깅 off  
    log_experiment=False,  
  
    n_jobs=1,  
    verbose=True  
)
```

(4) ML 기능 - 분류모델

For Cold Start User

```
best_models = compare_models(n_select=3, sort='F1')
```

F1 스코어를 기준으로 상위 3개의 모델을 선정

	Model	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC	TT (Sec)
lda	Linear Discriminant Analysis	0.7056	0.0000	0.7056	0.7077	0.7063	0.6101	0.6103	15.3680
xgboost	Extreme Gradient Boosting	0.7066	0.9183	0.7066	0.7121	0.7063	0.6167	0.6184	50.7400
lightgbm	Light Gradient Boosting Machine	0.7046	0.9185	0.7046	0.7111	0.7042	0.6145	0.6164	42.2680

(4) ML 기능 - 분류모델

For Cold Start User

```
blended_model = blend_models(estimator_list=best_models, optimize='F1', method='soft')
```

18	Fold Generator	StratifiedKFold
19	Fold Number	10

StratifiedKFold = 10으로 수행

	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC
Fold							
0	0.7110	0.9160	0.7110	0.7127	0.7117	0.6186	0.6187
1	0.7149	0.9175	0.7149	0.7158	0.7152	0.6237	0.6238
2	0.7174	0.9179	0.7174	0.7186	0.7179	0.6268	0.6269
3	0.7190	0.9196	0.7190	0.7199	0.7194	0.6287	0.6288
4	0.7163	0.9187	0.7163	0.7176	0.7169	0.6253	0.6253
5	0.7124	0.9163	0.7124	0.7134	0.7128	0.6201	0.6202
6	0.7168	0.9183	0.7168	0.7183	0.7174	0.6257	0.6258
7	0.7134	0.9174	0.7134	0.7147	0.7138	0.6217	0.6218
8	0.7197	0.9199	0.7197	0.7212	0.7203	0.6298	0.6299
9	0.7182	0.9182	0.7182	0.7200	0.7188	0.6285	0.6287
Mean	0.7159	0.9180	0.7159	0.7172	0.7164	0.6249	0.6250
Std	0.0028	0.0012	0.0028	0.0028	0.0028	0.0036	0.0036

(4) ML 기능 - 분류모델

For Cold Start User

```
pull_df = predict_model(blended_model)
report = classification_report(pull_df['Cluster_K5'], pull_df['prediction_label'])
```

▶ 성능 평가 (Hold-out set)...

	Model	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC
0	Voting Classifier	0.7172	0.9188	0.7172	0.7186	0.7177	0.6266	0.6267
		precision	recall	f1-score	support			
	0	0.73	0.71	0.72	40540			
	1	0.81	0.80	0.80	33032			
	2	0.72	0.72	0.72	9281			
	3	0.61	0.62	0.61	15887			
	4	0.65	0.68	0.66	21460			
	accuracy			0.72	120200			
	macro avg	0.70	0.71	0.70	120200			
	weighted avg	0.72	0.72	0.72	120200			

(4) ML 기능 - 분류모델

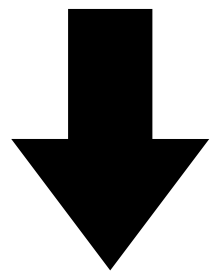
For Member(MYData)

	Model	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC	TT (Sec)
xgboost	Extreme Gradient Boosting	0.9624	0.9986	0.9624	0.9624	0.9623	0.9502	0.9502	142.2740
lightgbm	Light Gradient Boosting Machine	0.9546	0.9979	0.9546	0.9547	0.9546	0.9399	0.9399	112.4850
gbc	Gradient Boosting Classifier	0.9284	0.0000	0.9284	0.9289	0.9283	0.9051	0.9052	3513.6380
et	Extra Trees Classifier	0.9265	0.9941	0.9265	0.9271	0.9265	0.9026	0.9027	164.2420
rf	Random Forest Classifier	0.9228	0.9940	0.9228	0.9232	0.9224	0.8975	0.8977	335.4410

(4) ML 기능 - 분류모델

For Member(MYData)

```
tuned_xgboost = tune_model(
    xgboost_base,
    optimize='F1',
    n_iter=20,
    choose_better=True,
    verbose=True
)
```



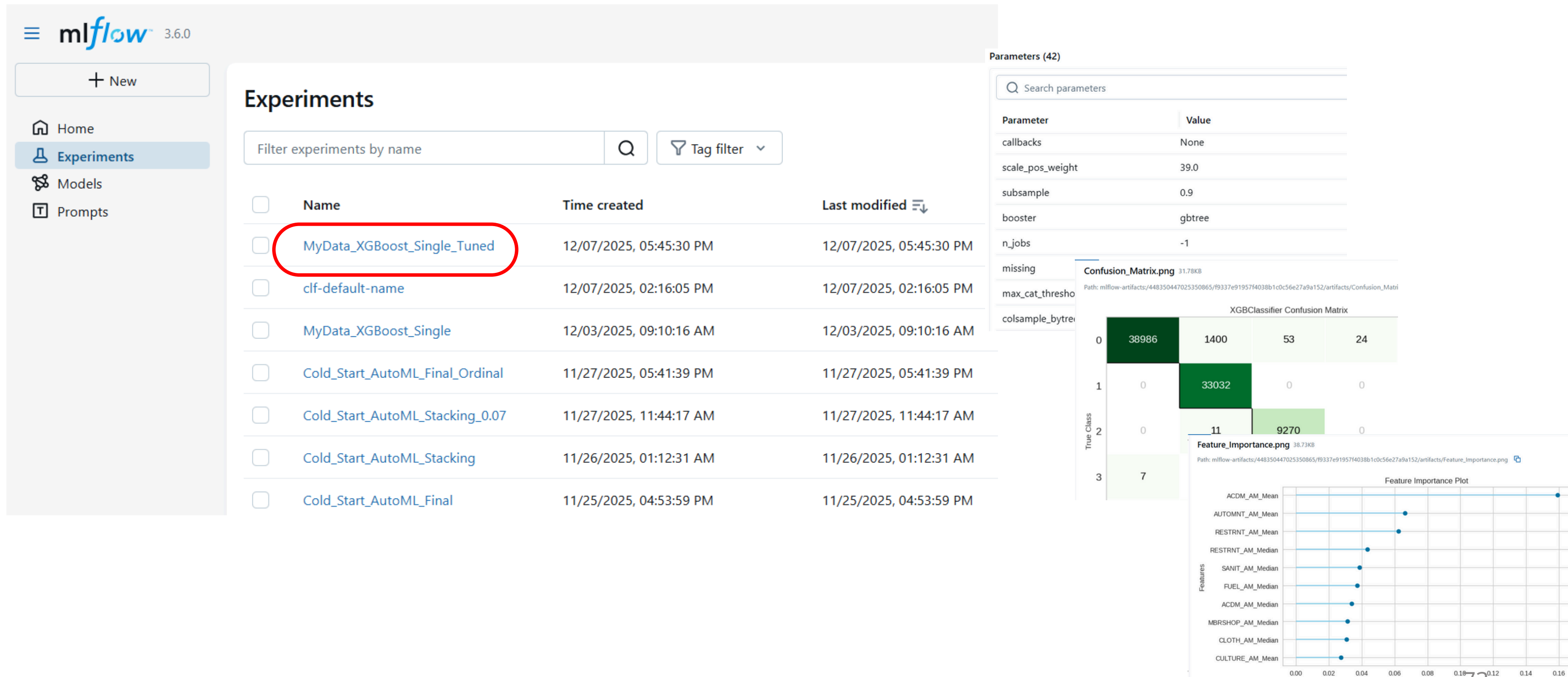
learning_rate → 0.4
max_depth → 7

▶ 성능 평가...

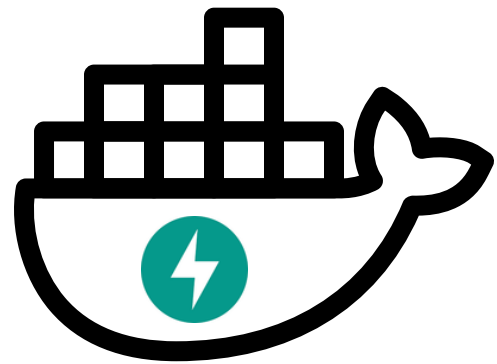
	Model	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC
0	Extreme Gradient Boosting	0.9692	0.9991	0.9692	0.9692	0.9691	0.9592	0.9592
	precision	recall	f1-score	support				
	0	0.97	0.97	0.97	40540			
	1	0.98	0.98	0.98	33032			
	2	0.96	0.97	0.96	9281			
	3	0.97	0.94	0.95	15887			
	4	0.96	0.96	0.96	21460			
	accuracy			0.97	120200			
	macro avg	0.97	0.97	0.97	120200			
	weighted avg	0.97	0.97	0.97	120200			

(4) ML 기능 - MLflow

```
experiment_name = 'MyData_XGBoost_Single_Tuned'  
mlflow.set_experiment(experiment_name)
```

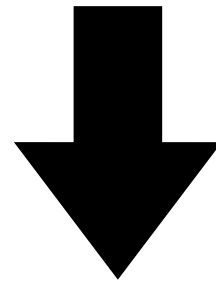


(4) ML 기능 - Model API



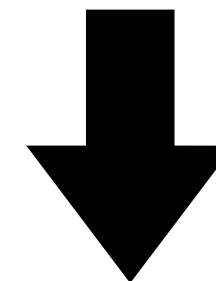
/predict → 설문응답을 받아 고객을 분류

/predict/mydata → 마이데이터로 분류



두 예측 엔드포인트가 실행될 시 DB에 예측 결과가 로깅
입력변수, 사용모델, 모델출력, 신뢰도

/feedback → 고객의 피드백 응답



해당 로그에 고객의 피드백 칼럼이 업데이트
고객이 생각하는 정답, 의견

→ 데이터 드리프트 탐지, 모델 재학습

ML 영상 - 콜드스타터-> 회원가입 -> 마이데이터 카드 추천

←

5 / 10



**본인 소유의 차량을
직접 운전하시나요?**

주유 할인이나 정비 혜택이 필요한지 확인해요.



네, 운전해요
월 주유 5만원 이상





아니요
대중교통 이용



다음으로 →

서비스 구성 및 구현 -메인 기능 소개

AIRFLOW

(1) Airflow를 사용한 기능 소개



알림

아침 9시까지의 사용금액
기준으로 각 카드 실적 달
성 여부를 알립니다.



데이터 수집 파이프라인

카드 데이터 수집, 전처리,
DB 저장 의 데이터 파이프라
인을 진행합니다.



(2) Airflow - 알림

사용 기술



Airflow



MySQL



Google Cloud Platform

get_user_id

@task

✓ success

send_email_via_api

@task

✓ success

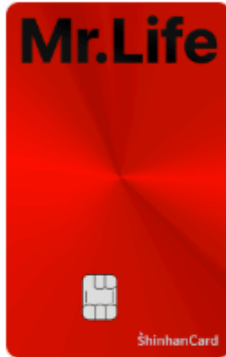
Airflow Dag

기능 화면

2 of 2,003

11:06 AM (6 hours ago)

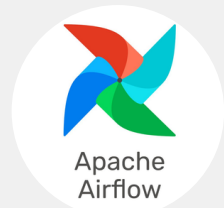
안녕하세요. 이번 달 카드 사용 실적을 안내해 드립니다.

카드 이미지	카드 이름	실적 기준	현재 사용액
	신한카드 Mr.Life	300000원	428734원
	삼성카드 taptap O	300000원	15401원

Email 알림

(2) Airflow - 알림

사용 기술



Airflow

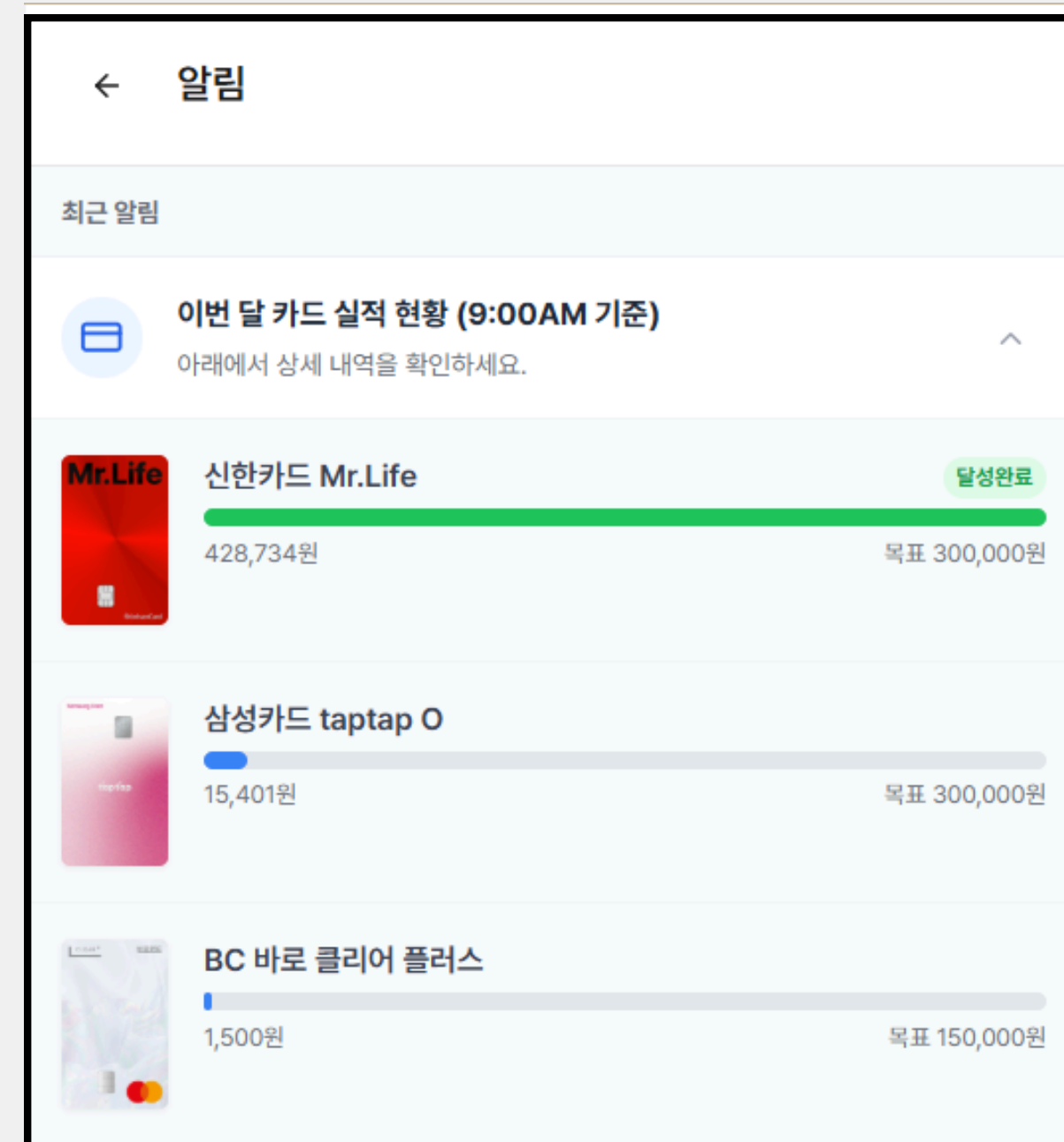


MySQL



Google Cloud Platform

기능 화면



앱 내 알림

get_user_id_list

@task

✓ success

update_To_DB

@task

✓ success

get_section

@task

✓ success

Airflow Dag

(3) Airflow - 데이터 파이프라인

사용 기술



Airflow



MySQL



Selenium

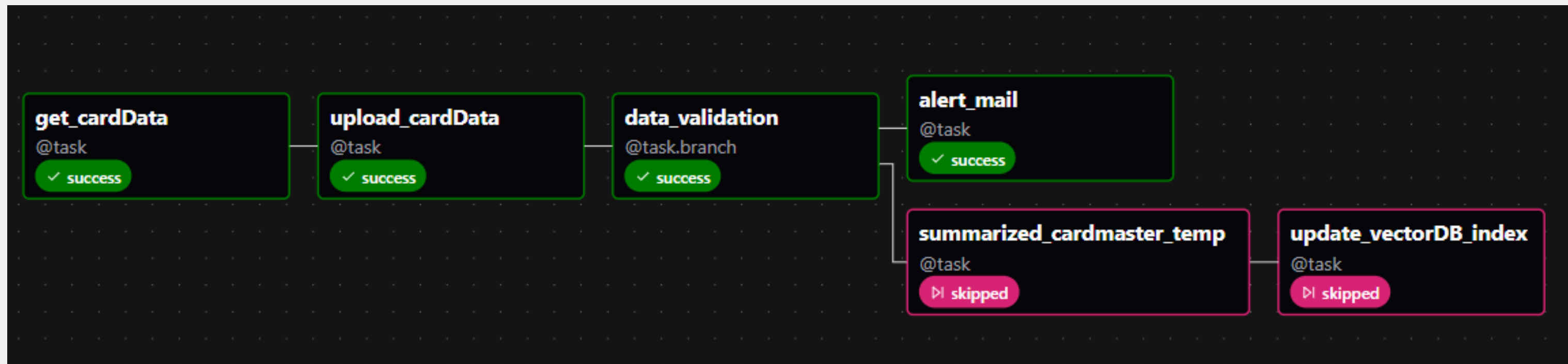


OpenAI

기능 화면

card_id	card_name	summarized_notice
1	신한카드 Hi-Point	[{"content": "카드 전체 유의사항 없음"}]
2	신한카드 Love	[{"content": "카드 전체 유의사항 없음"}]
3	신한카드 Lady	[{"content": "카드 전체 유의사항 없음"}]
4	SK에너지 신한카드The you	[{"content": "카드 전체 유의사항 없음"}]
5	GS칼텍스 신한카드Dream 100	[{"content": "카드 전체 유의사항 없음"}]

저장된 크롤링 데이터



Airflow Dag

05

프로젝트 결론 및 기대효과

- 5-1 ————— 기대 효과
- 5-2 ————— 트러블 슈팅
- 5-3 ————— 프로젝트 회고
- 5-4 ————— 참고 자료

기대 효과



사용자의 카드 혜택 활용률 상승



사용자의 비용 절감 및 만족도 향상



카드 사용 활성화 → 휴면카드 감소



디지털 취약계층의 혜택 접근성 개선



카드사의 마케팅 효율 및 브랜드 충성도 증가

트러블 슈팅

- 챗봇의 낮은 정확도의 답변

- Summary LLM을 통해 혜택 정제

공통 :

- 질문 : GS25에서 128000원 쓸거야
- 모델 : openai/gpt-oss-120b

사용 context	유의사항 원본	AI 요약본	성능
추천 카드	신한카드 Mr.Life	신한카드 Mr.Life	동일
할인 금액	1000원	1000 원	동일
시간	5.6 초	3.05 초	45.5% 감소 ↓
사용 토큰	6990 token	4937 token	29.37% 감소 ↓

트러블 슈팅

- **[Problem: 하드코딩의 늪]**
 - localhost 직접 입력 -> 배포 시 수정 불가
 - 반복되는 Header 설정 코드 -> 생산성 저하

트러블 슈팅

• [Solution: Axios 모듈화 & 환경변수 도입]

- api.ts로 인스턴스 중앙 관리
- Interceptor로 토큰/에러 자동 처리

```
// 2. 요청 인터셉터 (Request Interceptor)
// API 요청을 보낼 때마다 저장된 토큰(token)을 헤더에 자동으로 붙인다
api.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem('token');
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => {
    return Promise.reject(error);
  }
);
```

api 폴더

```
▼ api
  TS agentClient.ts
  TS analysis.ts
  TS auth.ts
  TS card.ts
  TS chat.ts
  TS client.ts
  TS ml.ts
  TS notification.ts
  TS point.ts
  TS transaction.ts
  TS user.ts
```

hooks 폴더

```
▼ hooks
  use-mobile.tsx
  TS use-toast.ts
  useAnalysis.tsx
  TS useAuthQueries.ts
  TS useCardDetail.ts
  TS useChat.ts
  useML.tsx
  TS useMyCards.ts
  useMyData.tsx
  TS useNotifications.ts
  TS usePayment.ts
  TS userTransactions.ts
  TS useUser.ts
```

예상치 못한
QA 시간

기록의
중요성

우선순위
관리의
중요성

환경/배포
이슈가 일정
에 미치는
영향

데이터
정합성이
성능을 좌우

- 랭체인과 RAG로 배우는 실전 LLM 애플리케이션 개발 - 양기빈 외 3인 | 위키북스
- 가장 빠른 풀스택을 위한 Flask & FastAPI - Dave Lee | 비제이퍼블릭
- Do it! LLM을 활용한 AI 에이전트 개발 입문 - 이성용 | 이지스퍼블리싱
- RAG 마스터: 랭체인으로 완성하는 LLM 서비스 - 브라이스 유 외 3인 | 프리렉
- LangGraph Documentation. LangChain AI.
- Model Context Protocol (MCP) Specification. Model Context Protocol.
- k-nearest neighbor (kNN) search. Elastic.
- Apache Airflow Documentation. The Apache Software Foundation.

THANK YOU

감사합니다.